

GEODE: A Growing Network of Frozen Models

Composable contribution, verifiable answers, and payment by measured use

Moazzam Abdullah Khan

August 27, 2026

Abstract

Artificial intelligence is in an arms race. Capabilities are built in secret and deployed in haste. They consolidate in a few hands, because the race rewards secrecy over safety. GEODE (Generalized Encoders for Open-Domain Expertise) tries to change that game. It is a network where anyone can contribute a capability and anyone can buy answers, priced in Ethereum at market rate. A capability is a frozen neural network (weights that no longer change) or a small deterministic program called a primitive. Capabilities are composable and routable. A contribution can be reused and built upon by anyone. Every use pays the work beneath it. Payment follows the measured work that actually served the query. Every decision is deterministic, replayable, and recorded on a hash-chained ledger. The wager: collaboration is the fastest path to advanced AI. Tasks break into smaller composable pieces. They are cheaper to train and cheaper to run. Competitors build on each other's work and share the rewards by measured use. A network anyone can improve compounds faster than central labs that fossilize under secrecy and intellectual-property protection. Those who refuse to collaborate fall behind. The surplus attention goes to safety and a rollout that benefits everyone. GEODE's claim is the assembly: the composed-codes architecture, i.e. frozen representation artifacts composed by declaration on a versioned code bus, paid by measured downstream use. It is not a new learning algorithm; the parts are old and named.

Contents

1	Introduction	3
2	The actors	4
2.1	Terminology	5
3	Design principles	6
4	Our assumptions	7
5	The architecture	8
5.1	The system as a black box	8
5.2	The pipeline	10
5.3	The registry and the router	10
5.4	The ledger and the anchor	13
5.5	Proofs of computation	15

6	The protocol in detail	17
6.1	The task descriptor	17
6.2	The fingerprint	18
6.3	The head	18
6.4	Registration proofs	18
6.5	Admission, step by step	19
6.6	The challenge session	21
6.7	Failure handling	22
6.8	Serving verification	23
6.9	The ledger	27
6.10	A session, end to end	27
6.11	Settlement batching	28
7	Programmable primitives	28
7.1	The standard library	28
7.2	Third-party primitives and their isolation	29
8	The game theory	30
8.1	Currency and pricing	30
8.2	The fee flow	31
8.3	The development fund's end state	33
8.4	Voting weight: pedigree gates, earnings scale	33
8.5	Registration	35
8.6	Slashing: burn, graded, replay-gated	35
8.7	Quorum takedown	36
8.8	Content orders and the authority-key registry	38
8.9	Bootstrap and the headroom rule	39
8.10	Coercion resistance	39
8.11	Primitive royalties	40
8.12	Who earns what	41
9	What has been measured	42
10	Prior art and position	45
11	Known limits	48
12	The methodology	52
13	Conclusion	53
A	Worked scenarios	53
A.1	Registering a new arm	53
A.2	The challenge session, from a validator's seat	54
A.3	Buying an answer	55
A.4	A probed session, honest path	55
A.5	Serving a substitute: caught	55
A.6	Probe-dodging attempts	56
A.7	A disputed mismatch: contributor vindicated	56

A.8 A fabricated mismatch: executor exposed	56
A.9 Becoming a reference executor	57
A.10 A third-party primitive, end to end	57
A.11 The quorum takedown	57
A.12 A content order arrives	58
A.13 The genesis council sunsets	58
A.14 The bootstrap handover	58
A.15 A wash ring tries and fails	58
A.16 A ledger rewrite attempt fails	58

1 Introduction

The buildup of artificial intelligence is an arms race. Participants build capabilities in secret, because secrecy is rewarded. Whoever ships first captures the returns. Whoever openly publishes the weights of their model early hands their work to competitors. The race has real costs. Secrecy defeats inspection. Systems ship with less understanding than their power warrants. Effort is spent duplicating work that already exists. Safety is a cost nobody can afford to pay first. And the race is winner-takes-all. The first actor to reach general capability would hold a technology too powerful to be governed by one hand. Whoever controls the most capable system sets the terms on which everyone else has to access intelligence itself. A race whose prize is that concentrated cannot be won safely by anyone. It can only be defused by changing what winning means.

GEODE changes the payoff structure. It does not preach against the race. Its mechanism is composability and routability. Capabilities are registered once. Then anyone reuses them and stacks them into chains [12]. Every use pays the work beneath it. This includes uses by competitors building on top. A *task* is a well-defined piece of work: “what object is in this picture?”, “transcribe this recording”, “forecast this signal one step ahead”. A *contributor* is anyone who offers a way to do such tasks. A *user* is anyone who pays for an answer. Research has always worked this way. Each generation stands on the shoulders of the previous one. GEODE’s wager: collaborative contribution is the fastest path to advanced AI, and composition is how it wins. Complex tasks break into smaller, more manageable pieces. They are cheaper to train and cheaper to run. Competitors build on top of each other’s work and share the rewards according to use. The effect mirrors the growth of cryptocurrencies. Innovation happens at the edges, not at the center. Anyone can make improvements, and everyone is incentivized. Centralized systems are difficult to change under secrecy and intellectual-property protection. They fossilize over time. Anyone who does not take this path loses out in the long run. If the best return on a capability comes from making it reusable, self-interest and collaboration point the same direction.

One thing makes it all possible. It is what the name stands for. GEODE is short for Generalized Encoders for Open-Domain Expertise. The idea is a registry of frozen encoders. Each maps one kind of input, whether image, audio, text, or number series, into a code space, and the registry composes them: a head declares a code manifest, an ordered list of the blocks it reads, and the blocks are concatenated into one design. CLIP [16] and ImageBind [17] built shared embedding spaces for single models. GEODE extends the idea from single models to a registry of measured tasks. A task plugs into the registry the way an application plugs into an operating system. A closed-form head is fitted on the composed codes. It is measured, priced, and routed on its own. Nothing beneath it is rebuilt. The registry does not need to be complete at launch. It needs to be general enough that new tasks enter as measured additions, not as new foundations. Where no registered encoder yet covers a data space, the registry admits another frozen encoder alongside

the first, and a head’s manifest simply lists both. Optional closed-form alignment artifacts, the Procrustes and CCA maps, may be registered as bridges between two blocks; each map is an exact solve, measured and priced per pair, and nothing in the protocol requires one. The measurements in this paper are the small, real version of that idea.

Three properties make that payoff structure possible. Everything else in this paper follows from them.

- **Frozen.** Every model in GEODE is a frozen publisher checkpoint. Its weights are fixed forever. A closed-form fit sits on top: one direct linear-algebra solve, no iterative training, no optimizer, no random seed. It is like fitting a straight line through points, in high dimensions, in one exact computation. Nothing learns while the network runs. The system reads its models and never writes them.
- **Replayable.** The same declared task and the same input produce the same routing decisions and the same answer. The route and the record replay from the hash chain for anyone. The artifact replays for whoever holds it. The validators check the replayed results. Weights need not be open for the decision to be auditable. Every decision is written to a hash-chained ledger. Records cannot be changed after the fact without breaking the chain.
- **Paid by measured use.** Payments are made in Ethereum at market rate. They flow to the contributors whose work actually served the query. They are released gradually over time. A small share goes to a development fund. Contribution is measured on held-out data. It is never self-reported.

GEODE’s ambition is modest and specific. It is to make inference *inspectable*: you can verify what served you and why. It is to make inference *accountable*: the people whose work serves you are the people who get paid. We do not train foundation models. We do not claim to beat the state of the art. We assemble established parts under one measurement discipline: mixture of experts, classical image codes, closed-form fits, Bulletproofs, and microVMs. We test the economic mechanism before it ever governs real money. Appendix A walks those rules through worked scenarios, actor by actor. Each scenario includes the abuse attempts and exactly where each one is closed.

One honesty statement organizes the rest of this paper. The protocol machinery is built and measured: the registry, the router, the probes, the ledger, and the settlement. Its rules are shipped code with sealed tests, and its numbers are held-out measurements. The generalized-encoder thesis is younger than the machinery: the fusion, nonlinearity, and breadth measurements in Section 9 support the federated form of the thesis with measured boundaries, but the thesis remains an open experimental program, not a closed claim. Where a measurement fails, the failure is reported. This paper contains its own negatives, and the reader should trust the negatives as much as the positives.

2 The actors

The system has few roles. Every rule in this paper attaches to one of them.

User. Anyone who pays for answers. The user declares the task and pays per unit of work at the posted price.

Contributor. Anyone who registers a capability. A capability is an arm (a frozen model) or a primitive (a small deterministic program). The contributor receives its usage fees, epoch-vested (an epoch is seven days). Both kinds share one registration form: an operator key, a

payout address that may differ from it, a price per unit of work, and a sealed claim. The kinds differ only in who executes them.

Host. The address that executes a primitive. The host earns the host share of its fees. Arms have no separate host role. Serving the arm is the contributor’s own obligation. It is measured by availability. Hired hardware is a private arrangement, invisible to the network.

Validator. A registered party that proposes challenges, scores answers, and audits other validators. It earns per accepted challenge.

Reference executor. A registered party that holds the sealed artifact and re-runs it inside probed sessions. It is sampled per session from the artifact’s pool under the validator eligibility rules (activation window, activity floor, tenure). It is structurally excluded from any artifact that serves the session. It earns a share of the registered reference-run price. The reference executor exists only for artifacts whose contributor chooses to distribute the sealed weights. For a weights-private artifact there is no serving-time identity check beyond behavioural identity on a fixed probe set, and the residual is priced by the bond.

Librarian. The role that records credits and files the replay quorum’s verdicts, the two acts that still need a named party. The settlement around them is not a privilege: anyone may incorporate an entry, post a root, deliver a branch, or claim, and the reward follows the work. It is an operator key, named at bootstrap by the developer’s key and at maturity by the governance executor, a keyless contract with no human key that holds that one power and no other.

Developer. The operator of the bootstrap. It holds the librarian key initially. It ships the bootstrap arms at 80–90% of capability and earns hosting fees only. Then it retires per the headroom rule (Section 8.9).

Development fund. The account that receives the 2.5% share and the registration fees. Its end state is the zakat rule (Section 8.3).

There is no adjudicator role. Any party may file a dispute with a deposit. The computation decides guilt. The computation is a replay of sealed data.

Two actors from outside the system matter as inputs. The *publisher* provides the frozen checkpoints the arms are built on. The *benchmark* is the public test suite used as context. Neither is a GEODE role. Neither is paid by GEODE. A third outside input is the *registered authority key*: a government key accepted through the multi-channel rule below. It can trigger a freeze. It can never burn, and it holds no other power.

2.1 Terminology

Every load-bearing term is defined here once and used consistently throughout.

Term	Definition
Capability	A contribution that can be served: a learned arm or a programmed primitive.
Artifact	A capability once sealed: frozen code, sealed claim, measured profile.
Arm	A learned contribution: encoder and head over a frozen checkpoint.
Primitive	A small deterministic program with typed inputs and outputs.
Block	The unit the encoder emits for a patch of input.
Representation artifact	The frozen block-builder the encoder is pinned to.
Head	The final projection from blocks (or codes) to scores.
Trunk	The frozen encoder the head reads from.
Encoder	The block-builder, trunk and head together.
Code	The block vector the encoder emits for one input.
Manifest	The hashed registration payload naming the encoder, its data, and the head.
Bus	The typed input/output channel a capability serves on.
Axis	One market: input type, output type, metric, floor, and privacy tier.
Epoch	One seven-day settlement period.
Promise	The unvested credit balance: attributed but not yet vested.
Beacon	The public randomness source every sample derives from: the beacon chain’s RANDAO composed with a verifiable delay function, with drand as the fallback, seeded as $H(\text{drand} \parallel \text{RANDAO-VDF})$ (safe if either source is honest).
Open probe exposure	The window during which a session’s answer is still subject to verification: a probe may still be drawn on it and the mismatch stream may still accumulate. Its length in work units is the detection horizon, i.e. the probed sessions the substitution test needs, divided by the probe rate. Accrued credits stay unclaimable for the exposure’s duration, which is also the per-axis bond’s sizing window and the Level-1 claim-freeze duration.

3 Design principles

The system’s behavior follows from a small set of rules.

Frozen. Publisher checkpoints, fixed dictionaries, and closed-form heads. No component changes at serve time.

Exact where possible. The task head is a closed-form ridge fit. Exactness removes the lottery of random-seed training runs.

Deterministic. Routing is a deterministic function of the declared task and the measured capabilities. Determinism makes replay possible. Replay makes audit possible.

Weights may be private. Verification rests on frozen hashes, measured reputation, and replay. It does not rest on reading weights. Contributors are never required to publish their artifacts.

The private serving path (below) keeps it that way even at inference time: no party holds the head in plaintext except the contributor’s own host.

Private serving. No party processes request content except the user’s device and the contributor’s serving host, and under the private path the contributor evaluates only ciphertext: the device encrypts the input, the host computes the answer on the ciphertext, the device decrypts. The developer and the network operator process metadata only (registry, ledger commitments, routing, and sampling), never request content. That separation is a design invariant, not a policy choice.

Measured, not asserted. Every capability claim is scored on held-out data. It is sealed with a content hash. It replays from the ledger. The discipline is register-before-measuring: a claim is written down before the measurement that tests it.

Economic-only incentives. Anti-abuse mechanisms use prices, delays, burns, and time-earned pedigree. There is no human-identity verification, no KYC, and no whitelist of participants; identity data is not collected. Pedigree is a time cost, not an identity check. It is earned through the activation window, the activity floor, and verified work. A fresh key is cheap and a pedigree is not, which is what makes the anti-Sybil layer work.

No control surface. The network has no user-selection path. A coercer can point at artifacts, never at audiences. The only content powers are public-record votes and authenticated freezes, and their effects are fixed: suspend, never burn. The protocol does meter per payer, and a payer address is pseudonymous; the metering is enforced locally at the gateway that a user connects to, and nothing per payer is ever written to the public ledger.

Earned, capped votes. A vote counts only with pedigree: the activation window, the activity floor, and verified work. It weighs only with unclaimed earnings. No identity’s weight may exceed a fixed share of the total.

Humble scope. Every number in this paper is small-scale and held-out. We state its limits alongside it.

4 Our assumptions

The design rests on the assumptions below. Each one is a stated belief, not a measured fact. Where this paper leans on established crypto practice, it says so in place. Two examples: paying for access instead of identity checks, and the limits of majority honesty.

1. **Privacy.** People and institutions will pay for answers whose processing respects their data. Inputs are not retained beyond the session. They are never used to train any component without the user’s consent.
2. **Verifiability.** People and institutions will pay for answers they can verify. Verification rests on the artifact being frozen, hash-identified, and reputation-scored. It also rests on the decision record replaying from the hash chain. None of this requires weights to be open.
3. **Composition (a): fusion.** Several frozen encoders can be concatenated on one code manifest and read by a single head without the blocks fighting each other. Both composition forms are measured: fusion here, chaining in the next item. Two full-scale multi-encoder fusion cells are sealed. The first cell, multi-scale codes plus DINOv2 features upsampled from 32×32 , is

a negative for both plain concatenation and CCA alignment, with the upscaling confound registered rather than hidden. The second, on a clean cell where both blocks carry signal (the DINOv2 block re-extracted at native resolution), reverses the concatenation reading: the fused design more than doubles the single encoder (0.548 vs 0.242), while CCA alignment still loses to raw concatenation (0.511). Concatenation on the code manifest is the load-bearing fusion form; alignment bridges are optional, measured per pair. Both outcomes are published.

4. **Composition (b): chaining.** The core concept is a pipeline: capabilities link so that one stage’s output becomes the next stage’s input, and the linked stages together serve a task none of them serves alone, e.g. transcribe the recording and then read the intent out of the transcript. A chain $g \circ h$ is admissible exactly when the output contract of h satisfies the input contract of g , written $C_{\text{out}}(h) \subseteq C_{\text{in}}(g)$, i.e. the outputs of the first stage must be valid inputs for the second. Chaining is a different claim from fusion, with different failure modes: concatenating blocks into one head tests neither the contract check nor the error a first stage hands to a second.

The form is tested end to end (Section 9): a domain router feeding six per-domain specialists on the sealed DomainNet selection. The chain reaches 0.2741 against 0.2450 for the monolithic head that is the strongest single stage, and 0.2995 when the router is handed the true domain. Routing error costs part of what specialisation earns.

The tested scope is narrow: one chain of length two, in which both stages read the same frozen features. Chains of length three and above, and chains that cross modalities, remain unmeasured. Where composition does not hold, an arm can be a large deep network at the current state of the art. The design does not depend on either path winning everywhere.

5 The architecture

5.1 The system as a black box

From the outside, GEODE takes two things in and gives one thing out.

- **In: a task description.** A declaration of the kind of work. It states the input type (image, audio, text) and the output type (class label, transcript, number). It also states a small set of fixed attributes. Each attribute comes from a short allowed list, never free text. An *axis* is a measured task type: input type, output type, metric, a published quality floor, and a privacy tier, fixed together. The axis metric is per-kind accuracy, $\text{pass}@k$, or word-error rate, computed restricted to the label set the buyer declares. The artifact’s measured coverage (the share of inputs it answers rather than refuses) is published beside the score and decides *qualification*: an arm that does not cover the declared set is not offered for it at all. That is what stops an arm answering a fifth of inputs at high accuracy from outranking an arm answering everything, without the coverage multiplication that used to invert the ranking in the other direction (Section 5.3). The routing mode is best-value or best-quality. Optional fields cap the unit price and the total spend. The task is declared with every request. The system never guesses it.
- **In: the sample.** The data to process: the picture, the recording, the sentence.
- **Out: a typed answer with a confidence, or an abstention.** The system prefers no answer to a wrong or unsafe one. It says so. This is the selective-classification stance of [13]. Either way, the decision is recorded. The answer carries a tag: the artifact that produced it.

That artifact’s fingerprint, what it accepts and what it produces, lets the next stage of a chain route onward.

The equations in this paper are shorthand for the sentences around them. Every symbol is explained in words beside it, and no rule in the system depends on the reader solving any of them.

On a classification axis with a closed-form head, the compute path is, in symbols, four operations. A frozen encoder maps the input to a code vector

$$z = f(x) \in \mathbb{R}^d. \quad (1)$$

The sealed head scores the C classes of the declared task as a fixed sum of products plus a bias,

$$s = W^\top z + b, \quad W \in \mathbb{R}^{d \times C}, \quad b \in \mathbb{R}^C, \quad (2)$$

with W the class projection and b the class intercept (the ridge fit’s bias, folded into W in any place that writes the head without it). The answer is the top-scoring class, the entry of s with the largest value,

$$\hat{y} = \arg \max_{j \in [C]} s_j, \quad (3)$$

Confidence is returned as a coarse bucket, never the raw number: the winning class’s softmax margin is compared against the artifact’s registered bucket edges, and the answer carries only the predicted class and the bucket index,

$$\kappa(x) = \text{softmax}(s)_{\hat{y}} - \max_{j \neq \hat{y}} \text{softmax}(s)_j, \quad b = \text{bucket}(\kappa(x)), \quad (4)$$

The edges are part of the sealed artifact, so the bucketing replays exactly. The raw margin and the score vector never leave the serving host: with a public trunk, a margin-annotated answer stream would let a buyer solve for the head as a linear system in $d \cdot C$ unknowns; the bucket leaves only coarse information, and the extraction-cost analysis in Section 9 prices what remains. The capability abstains whenever $\kappa(x) < \tau$, the axis’s registered margin threshold, a floor that each axis publishes. Number and transcript outputs are read out of the same vector through the task’s registered decoder; the four-step form is unchanged.

The contract check enforces the agreement. A sample that does not match the declared task is rejected, not interpreted.

Verification does not require open weights. The user knows three things about the capability. It is frozen. It cannot change under them. Its hash ties it to its record. Its reputation is published as measured held-out scores. How does the user know the computation was done correctly? How does the user know their data stays private? In layers.

The head is a fixed sum of products: the answer is the class with the largest score, $\hat{y} = \arg \max_j (W^\top z)_j$. On a sampled fraction of settlement batches, whoever computes it attaches a Bulletproofs-style argument. The argument proves that the reported answer is exactly what the sealed head computes. It is not cheaper than recomputing; for a linear head, nothing is cheaper. Section 5.5 prices it per axis. What it buys is that the proof reveals nothing about the input, and that the verifier never needs the private head. This is verification without disclosure.

The deep encoder is not covered by the proof layer. The reason is speed, not mathematics. Proving a large network is possible today. It costs orders of magnitude more than serving. The design deliberately does not depend on it. This is an honest boundary.

The encoder is a frozen checkpoint. A substituted model is caught probabilistically. A fraction of sessions is answered twice: once by the serving capability and once by a reference execution of the sealed artifact. This is the shadow probe. The probed contributor pays for the reference run.

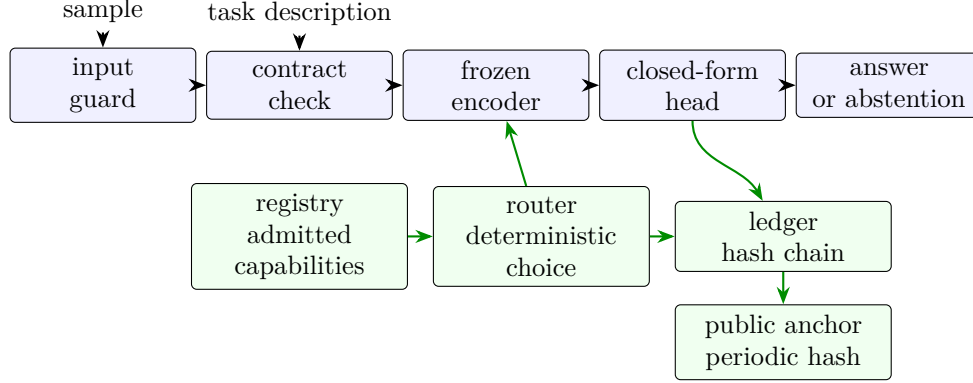


Figure 1: The GEODE architecture. The pipeline across the top (blue) turns a sample into an answer; the standing services beneath (green) select, record, and anchor every decision.

The answers must match exactly. The serving host commits its answer before it learns whether the session is probed. There is no way to be honest only when watched.

The user’s input enters no public record. The ledger records a commitment to the answer and the meter, never the answer itself. The commitment is the answer’s hash with a fresh nonce. The commitment form closes the transcript-axis exposure: there the answer is the content of the sample, and a commitment to it publishes no content. The opening exists only inside the sealed replay environment, where a dispute is settled. The data contract: inputs are not retained beyond the session. Codes are not retained either. A code is the encoder’s compact numeric summary of an input, and it can be inverted back toward the input by a feature-inversion attack. Neither inputs nor codes are ever used to train any component without the user’s consent.

5.2 The pipeline

Inside, one input walks one path. First, an input guard rejects inputs outside the known input space. Then the contract check runs. A frozen encoder turns the input into a code vector. A code vector is a compact numeric summary of the input. The closed-form head turns the code into the answer. An output contract checks the answer’s type and attaches confidence.

Standing services surround this pipeline. The registry lists the admitted capabilities. The router picks the capability for the declared task. The guards enforce the contracts. The ledger records every decision.

5.3 The registry and the router

Admission is by measurement, and the registry changes only through deterministic rules. The admission rules are:

- A submitted registration, an arm or a primitive, passes quality gates on held-out data. The claim is sealed before evaluation. This is commit-reveal. A bad result cannot be re-described after the fact.
- Quality is measured and sealed. The market prices only the cost side. A low posted price cannot buy a better score. A high one cannot claim it.

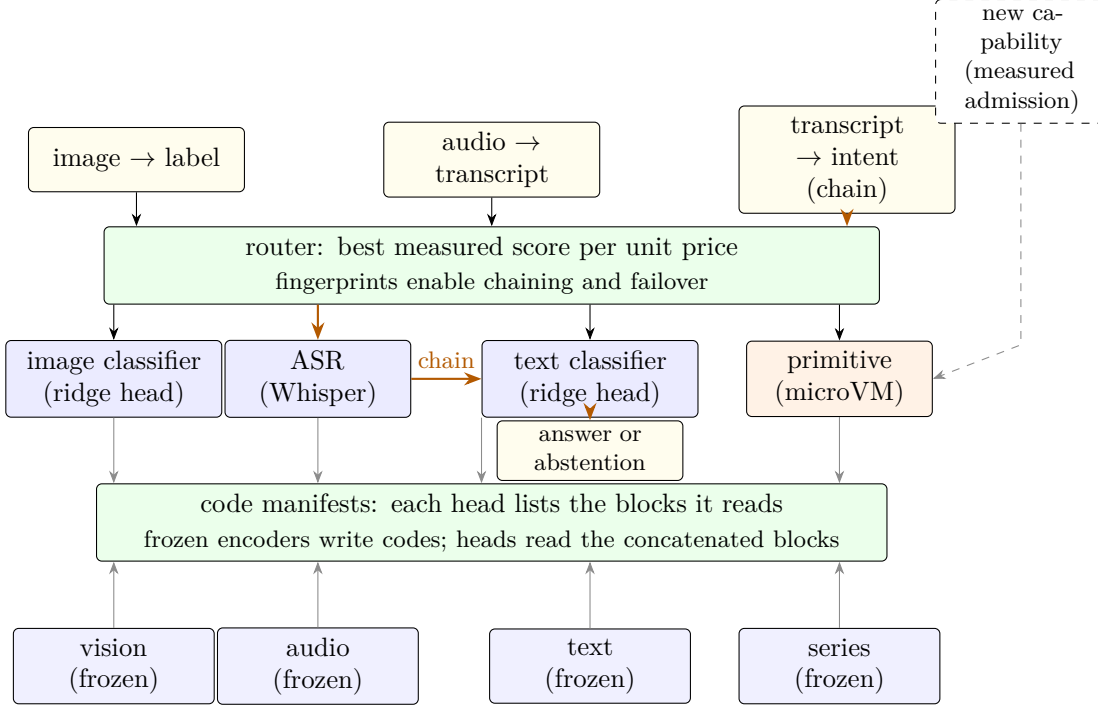


Figure 2: The network view. Frozen encoders (bottom row) each write one modality’s codes; a head’s code manifest lists the blocks it reads, and the blocks are concatenated into one design. Admitted capabilities answer tasks: arms with closed-form heads and primitives in microVMs. The router picks one capability per declared task. The orange path shows a chain: transcribe audio, then classify intent, each stage’s output satisfying the next stage’s input contract. New capabilities join through measured admission (dashed).

- New tasks grow the registry the same way. A measured task node appends its descriptor and fingerprint to the capability map. The capability map is the registry’s index of what each capability can do. The extension follows a deterministic rule.
- Feature blocks are versioned artifacts on a code bus: each registered encoder extraction carries an encoder version, an extraction version, a preprocessing version, and a content digest. A head declares a code manifest, an ordered list of the blocks it reads, and the head is replayable exactly when its manifest resolves. A version that does not resolve is a refusal, never a silent default. A new trunk is therefore a new bus entry, not a registry invalidation: old heads keep resolving their manifests, and new heads may read old and new blocks together.
- The registry updates are filed by any party under a bond. The librarian executes the replay quorum’s verdict as a clerk: it files the sealed measurement, the rule decides, and the key never admits or extends on its own discretion.

Routing is deterministic (Figure 2), and it follows these rules:

- The router ranks qualified capabilities by quality per expected charge: the measured score divided by the posted unit price and by the expected unit count of the axis’s reference workload. The score is the axis metric oriented so that higher is better: accuracy, pass@ k , or inverted word-error rate. Only capabilities that passed the axis’s admission verdict are considered. A capability that pads its output pays for it here: bloat inflates the expected charge and

lowers the ranking by exactly that factor. The discipline descends from mixture-of-experts routing [1, 2]. Replacing the learned gate with a fixed rule is itself established prior art: hash-based routing [3] and COMET’s fixed random projection with a winner-take-all cap [4] route without a trainable gate. What GEODE adds is that the fixed rule is public and replayable, and ranks independently registered frozen artifacts by measured quality per expected charge.

- The route is a weighted lottery over the top five, with weights equal to the ranking. It is not winner-take-all: every qualified capability in the top five keeps a share of traffic in proportion to its ranking, and the winners’ pool mates form the failover chain in rank order. A newcomer with a smaller model and a lower price enters the top five on efficiency, not on pedigree. On a synthetic sweep of the rule, the strongest arm held about one third of traffic (0.355), an equal-price tie split evenly (0.490 / 0.510), and a two-times price cut roughly doubled an arm’s share (ratio 1.95). Those figures require the per-session term in the seed: with the seed drawn from the beacon alone, the same sweep gives a single arm 1.000 of traffic in all three cases. Rank ties resolve by a hash of the artifact and the anchor, never by the bare artifact hash. The hash is unknowable at seal time and deterministic per anchor.
- A per-axis price floor sits at the reference hosting cost. Registration below the floor is rejected, and below-floor prices are never routed. The floor stops a price race to zero. It does not stop an efficient operator from pricing at the floor and taking the share that efficiency earns.
- A task may declare a best-quality mode. This mode ignores price. A task may also declare a max unit price. Capabilities priced above it are not considered.
- Posted prices are ledger entries. Every route replays against the price table of its day.
- If no capability is qualified, the system abstains and says so.

Formally, let $Q(t)$ be the set of admitted capabilities qualified for task t , s_a the held-out score of capability a on the axis, p_a its posted unit price, and $\bar{u}_a$ its expected unit count on the axis’s reference workload.

The score is conditioned on the buyer’s declared label set, never multiplied by coverage. Let D be the label set the task declares. A capability enters $Q(t)$ only if it covers D ; covering less of D is a *qualification* failure, not a ranking penalty. For those that qualify, s_a is the axis metric computed restricted to D , oriented so that higher is better, the single definition used everywhere in this paper. Abstentions are metered units, so they count in $\bar{u}_a$.

An earlier form of this rule multiplied the metric by coverage. It inverted quality, and measurably so: on the Open Images axis it ranked a scoped arm reading 0.901 on the classes it serves at $0.901 \times 0.049 = 0.044$, below a full-coverage arm reading 0.164 on everything. Best-quality mode, which takes $\arg \max_a s_a$, therefore returned the worse arm, and abstention was penalised twice over. Conditioning on D keeps the property the coverage factor was defending, that an arm cannot win by refusing everything, because an arm that refuses part of D never qualifies for D at all.

The selection score is the quality per expected charge

$$v_a = \frac{s_a}{p_a \bar{u}_a}, \quad (5)$$

where an unmeasured $\bar{u}_a$ counts as one and is flagged as such. Best-value routing draws the route from the top five capabilities by v_a , each weighted by its own v_a , so capability a is chosen with probability

$$\Pr[a \mid t] = \frac{v_a}{\sum_{b \in T_5} v_b}, \quad T_5 = \text{the top five of } Q(t) \text{ by } v_a. \quad (6)$$

The draw is seeded by the randomness beacon, the same composed source every other sample in the protocol draws from, ordered by the epoch anchor, together with the task, the registry state, the task fingerprint, and the session identifier. The session identifier is what makes it a lottery: the other fields are constant within a beacon round for a given task, and a seed built from those alone sends every session of that round to the same arm. The identifier is assigned at declaration and recorded in the ledger, so every route still replays exactly; it is not chosen by any host, so no host can farm its own draw. Best-quality routing skips the lottery and selects $r(t) = \arg \max_{a \in Q(t)} s_a$. In words: best-value routing spreads traffic over the top five in proportion to quality per expected charge; best-quality routing picks the single highest measured score. Capabilities are ranked only against others in the same privacy tier; a cross-tier route is refused. If $Q(t)$ is empty, the system abstains.

Crowding defense. The base rule is the router of Section 5.3: it ranks qualified capabilities by quality per expected charge $v_a = s_a/(p_a \bar{u}_a)$ and draws the route from the top five as a weighted lottery. A price floor is a gate, not a rank: five barely-passing arms priced at the floor would hold all five lottery slots and evict a better arm that prices above the floor, because quality never enters the ranking once an arm is through the gate.

Two additional rules are triggered in the event of crowding on-chain, and the axis registers which of them applies:

1. **Reserved slots.** The top five is the union of the top three by v_a and the top two by s_a . A dominant-quality arm can never be evicted by price alone. Membership changes; weights stay the base rule's.
2. **Quality exponent.** Rank on $s_a^\gamma/(p_a \bar{u}_a)$ with γ registered per axis; $\gamma = 1$ recovers the base rule. The exponent re-weights traffic toward the quality arms as well as admitting them.

Both additional rules are measured under the attack: in the registered scenario, a strong arm at 0.95 and a good arm at 0.85 priced at twice the floor, against a crowd of floor-priced arms at 0.60, the base rule serves only the crowd (delivered accuracy 0.600) once the crowd reaches five, while both additional rules hold the quality arms in the pool at every crowd size and deliver 0.701. An axis may switch between the rules with a governance registration, never by a code path; all three are measured and reported in Section 9.

5.4 The ledger and the anchor

The ledger is the network's hash-chained record, and the anchor notarizes its tip on a public chain. The rules follow:

- **The chain.** Every decision, be it a route, an answer, an abstention, or a payment event, is appended to a hash-chained ledger. Entry i commits to its payload and its predecessor:

$$h_i = H(\text{payload}_i \parallel h_{i-1}), \quad (7)$$

with h_0 the genesis hash. In words: each link seals its own contents and the seal of the link before it. Change the content and the hash changes. No record can be altered without breaking the chain.

- **The librarian.** One role records credits. The settlement around that append is not a privilege: incorporation, root posting, branch delivery, and claims are all permissionless, and

the reward follows the work. What is left to the role is writing the credits a session's evidence supports and filing the replay quorum's verdicts, which need an on-chain authenticator the network does not yet carry. It is an operator key, named at bootstrap by the developer's key and at maturity by the governance executor, a keyless contract with no human key. The role does not rotate on a schedule and posts no stake; replacement is its whole discipline, which is why the next two rules carry the weight they do.

Two separate authorities can name the librarian, and keeping them separate is what makes the replacement rule real. The developer's key names it during bootstrap and then closes. A governance executor names it thereafter, holds that one power and no other, and can hand itself on to a successor. A single authority would have to choose between two failures: keep the developer's key alive and the network never retires it, or close the developer's key and the librarian address freezes at whoever last held it, with a two-thirds vote deciding something it has no way to carry out.

- **The anchor.** By default, the ledger tip is anchored to Ethereum once per epoch. The cadence grows with traffic, because the cadence is the rewrite window. During development the anchor goes to the Sepolia testnet, a public practice chain whose coins carry no value. On mainnet it goes to Ethereum. The anchor is a notarized public record.
- **Prefix immutability.** A chain whose already-anchored prefix disagrees with any earlier anchor is invalid outright. Rewriting between anchors cannot be laundered by a later anchor. The fork rule applies only to unanchored suffixes. A divergence inside an anchored prefix is a recorded reason to replace the librarian.
- **Force-inclusion queue.** Any party may post an entry directly to the settlement contract, and *any party may incorporate the entry at the head of the queue*. Nobody has to be asked. A poster who is being ignored clears its own entry, so withholding a rival's registration, a dispute filing, or a mismatch record does not delay it at all. The queue is an on-chain inbox contract, and the state of it is queryable on chain and is never silent.

The reason incorporation needs no permission is that it decides nothing. Every step it takes is already fixed by what is on the chain: which entry is next, whether the deadline passed, what the bond returns. A rule that only one party may perform a calculation anyone can perform protects nothing, and costs the network a party it has to wait for. Where the paper still names a role for an action, that action turns on evidence or on a vote, not on the calculation itself.

A poster pays two amounts. The bond is refundable: it returns on incorporation, and a poster whose entry is left to rot withdraws it without discharging the librarian's obligation. The posting fee is not refundable, and it *follows the work*. It is held against the entry until someone incorporates it. If the librarian does the job, the fee goes to the operations line. If the librarian lets the deadline pass and someone else clears the entry, the fee goes to whoever cleared it. A librarian that stalls does not simply fail to earn; it watches its income move to whoever covered for it, one entry at a time, with no vote and no penalty ruling in the path. The fee rises with the square of a sender's posts in one epoch past a small free allowance: posting a thousand entries costs about three hundred and thirty thousand times what a flat fee would charge, which is what stops a queue nobody can clear from being bought with gas alone. A poster that clears its own entry before the deadline does not get the fee back, or posting would be free again and that price would collapse.

Two further rules keep the queue from becoming a weapon. The librarian owes at most a registered number of incorporations per epoch; a backlog beyond that rolls forward, and

every entry still carries a deadline fixed and computable at the moment it is posted. And incorporation runs in posting order, so nobody can serve a friend ahead of a rival. Ordering the queue is also what keeps the validity check cheap: a validator reads the oldest open entry and nothing else, so an attacker cannot make the question itself expensive to ask. The contract records who incorporated each entry, which is a sharper statistic than a missed deadline: an entry a stranger had to clear is direct evidence about the librarian.

- **Executable replacement.** The librarian is replaced when a recorded divergence reason collects endorsements carrying two thirds of the earned voting weight, under the same pedigree gate, twenty percent cap, and diversity floor as every other governance vote; a deterministic deputy takes over at the next epoch. The trigger stays mechanical: a recorded divergence reason is a fact a validator can replay, and no amount of weight substitutes for it. Only the endorsement is weighted. Replacing the role that appends every ledger entry is the most powerful action the network can take, and counting registered validators by head would have priced it at the cost of registering identities.

The force-inclusion queue reads the librarian’s address from the ledger rather than storing its own copy, so a replacement reaches the contract that contains a misbehaving librarian at the moment it reaches anything else. The deputy inherits the open queue. An entry someone is being censored over survives the replacement it may well have caused.

- **Liveness statistics.** Anchor cadence and inclusion latency are measured, public statistics. A librarian that stops anchoring, or sits on the inclusion queue, is visible to everyone on the ledger, not just to its victims.
- **Settlement.** Payments are finalized on Arbitrum One. Arbitrum One is a rollup. It executes transactions cheaply and posts a compressed proof of them to Ethereum.
- **The asset.** ETH is the native asset of both chains. Payment and claim stay on one asset with no wrapper.

5.5 Proofs of computation

An individual answer does not carry a proof. A sampled fraction of settlement batches does. That placement is forced by cost, and the cost is stated here rather than implied.

The task head is a closed-form ridge fit. It is a fixed sum of products. Whoever computes it can prove, with a Bulletproofs-style inner-product argument, that the reported answer is exactly what the sealed head computes [14]. In symbols, the prover publishes commitments to the sealed head W and the input’s code z and a compact proof π of the inner-product statement

$$s = W^\top z, \tag{8}$$

of size logarithmic in the vector length; the proof grows slowly, even as the head grows.

What the proof buys, and what it does not. It does not buy cheap verification. For a linear head, redoing the computation is the cheapest operation in the system, and checking a proof costs far more than recomputing. What it buys is verification *without disclosure*: the verifier cannot redo the computation, because W is private. That is the whole benefit, and it is enough where the head must stay secret.

The table gives the measured cost per registered axis on the reference CPU. Serving is a float64 mat-vec of $n = dC$ multiply-adds. Prover and verifier costs are the analytic Bulletproofs operation counts priced at a measured ed25519 scalar multiplication of $152.3\mu s$, with multi-exponentiation costed by the bucket method, which is the model most favourable to the proof system.

Axis	$n = dC$	Serve	Prove	Verify	Proof
Text, SST-2 (768×2)	1,536	1.2 μ s	0.09 s	0.034 s	1.1 KB
Text, MNLI-m (768×3)	2,304	1.4 μ s	0.13 s	0.048 s	1.2 KB
Audio, SC-v2 (768×35)	26,880	4.2 μ s	1.18 s	0.434 s	1.4 KB
Routing (1024×345)	353,280	35.8 μ s	12.80 s	4.615 s	1.6 KB
Vision (1024×601)	615,424	102.0 μ s	21.48 s	7.721 s	1.7 KB

On the vision axis the verifier costs about seventy-five thousand times a recomputation and the prover about two hundred thousand times. Granting a hypothetical implementation ten times faster than the one measured moves that to twenty thousand, which changes nothing about the placement. The $O(\log n)$ size claim holds and is a size claim only: proofs stay between 1.1 and 1.7 KB across a four-hundred-fold range in n . No claim about the cost of these proofs appears anywhere in this paper that is not in this table.

Two placements follow.

- **Settlement batches.** Each settlement batch carries a hash of the proofs of the computations it pays for. That proof-hash is anchored on-chain with the batch. A payment is tied to a provable computation, not just a claim. The prover cost is a registered per-axis number and is priced into the fee flow; it is not free and is not hidden.
- **The verifier.** Anchoring a proof-hash without verifying the proof would make it decorative. A sampled, paid batch-verification step at settlement checks the proofs themselves. It is a registered, ledger-measured role. An invalid proof is a ledger-disputable L1 deviation. It is replay-gated. It burns the unvested promise of whoever submitted it.
- **Disputes.** A slash is replay-gated. A filed dispute re-runs the sealed artifact on sealed data. The re-run is itself a verifiable measurement. The computation decides guilt. Note that the re-run, not the proof, is what settles a dispute; for a head, replay is four to five orders of magnitude cheaper than proof verification.

The honest boundary: the proofs cover the small, exact components. They cover the closed-form head and the router. They do not cover the full encoder. Proving a large deep network is possible in principle. It is active research. It costs orders of magnitude more than serving, and the proof layer does not include it. GEODE claims no novelty in the proof technology itself.

Where each mechanism is available, per serving tier. The proof layer is not available on the private tier at all: the host never holds z in plaintext there, so it cannot produce an inner-product argument about $W^\top z$ without verifiable homomorphic evaluation, which is research-grade and multiplicative on top of an already impractical cost. Stating that in a matrix makes the gaps unhideable.

Tier	Shadow probe	Behavioural identity	Head proof	Dispute replay
Plaintext	yes	yes	yes	yes
Device encoder	yes	yes	yes	yes
Private (FHE)	residual	residual	no	residual

On the private tier every check degrades to a residual rather than a guarantee: the reference executor and the serving host both operate on ciphertext, so sameness can be checked only where the encoding is deterministic, and the head proof is unavailable outright. That tier is a registered research target with a cost trigger, outside the served tiers (see Section 11).

6 The protocol in detail

The rules above are restated here at implementation depth. An independent implementer can rebuild the system from this description. Where a parameter is adjustable, its adjustment path is stated with it.

6.1 The task descriptor

A task is a declaration of fixed fields:

- **Input type:** image, audio, text, or number series. The input type carries its input contract.
- **Output type:** class label, transcript, or number. The output type carries its type check.
- **Axis metric:** per-kind accuracy, $\text{pass}@k$, or word-error rate.
- **Privacy tier:** plaintext, device_encoder, or private. The tier is part of the axis identity, not a preference: arms compete only within a tier and a cross-tier route is refused. See Section 6.8.
- **Declared label set** (classification only): the classes the buyer cares about. The axis metric is computed restricted to this set, and an arm that does not cover it is not qualified for the task at all.
- **Unit of work:** derived, never chosen. The input and output types together determine the unit by the registered table below.
- **Routing mode:** best-value (default) or best-quality.
- **Max unit price** (optional): a per-unit price ceiling. Capabilities priced above it are not considered.
- **Max spend** (optional): a total cap on the session’s charges, in ETH. Serving stops when the remaining cap is less than one unit. Unspent budget is never charged.

The descriptor is content-hashed; its registry identifier is $\text{id}(\tau) = H(\tau)$, a fixed-length digest of the descriptor’s exact contents, and that digest is the task’s identifier in the registry.

How the unit follows from the description. The unit of work is a function of the input type and the output type together. It is not a function of either one alone. It is not a free field. A registered table maps each admissible pair to its unit:

Input	Output	Unit
image	class label	query
text	class label	query
number series	number	query
audio	transcript	audio second (input duration)
text	transcript	token
text	audio	audio second (output duration)
<i>any (primitive)</i>	<i>any</i>	execution attempt

A pair without a row is not an admissible task until the table is extended. The extension is a registered rule change, never a per-task choice. Describing the task fixes the unit. No registration chooses it. Every registration on the axis prices in it. The descriptor’s hash freezes it.

6.2 The fingerprint

A capability’s fingerprint encodes what it accepts and produces. It is the tuple

$$F(A) = (\tau, C_{\text{in}}, C_{\text{out}}, \mathcal{L}), \quad (9)$$

where τ is the task descriptor, C_{in} and C_{out} the input and output contracts, and $\mathcal{L}$ the class list, i.e. what task it serves, what inputs it accepts, what outputs it produces, and which labels it can emit. Two capabilities with identical fingerprints are interchangeable on that task. That makes composition and failover mechanical. The fingerprint is computed from the sealed artifact. It is never computed from a self-description.

6.3 The head

The head is a ridge fit on frozen features. There are n training rows with code matrix $X \in \mathbb{R}^{n \times d}$ and label matrix $Y \in \mathbb{R}^{n \times C}$. For classification, each row of Y marks the true class with a 1 and the rest with 0. That is a one-hot encoding. The head minimizes the penalized least-squares objective: the squared distance between the head’s predictions and the labels, plus a small penalty λ on large weights. It is the same calculation as fitting a straight line through points, in high dimensions,

$$\min_W \|XW - Y\|_F^2 + \lambda \|W\|_F^2, \quad (10)$$

whose single exact solution is

$$W = (X^\top X + \lambda I)^{-1} X^\top Y, \quad (11)$$

with λ a registered regularization parameter. At serve time a fresh code z is scored by the sum of products $s = W^\top z$ and answered by the class with the largest score, $\hat{y} = \arg \max_j s_j$. The solve has no iterations and no random seed. The solver, λ , and the training rows are part of the sealed artifact. The head replays bit-exactly. It is a closed-form readout [11] in one exact solve.

6.4 Registration proofs

A registration is accompanied by evidence about what the artifact computes, and the boundary of that evidence is fixed in advance. The rules are:

- **Contributors.** At registration the contributor submits the artifact hash, the fingerprint, and a contract proof. The proof is a compact Bulletproofs-style argument [14]. It proves that the sealed head satisfies the declared output contract on a sampled set of ledger-registered reference codes. A reference code is the frozen encoder’s output on a reference input, and the encoder run that produces each code is itself registered in the ledger, so the code is a recorded datum, not a raw input. For each sampled reference code z_i , the proof attests

$$y_i = \text{decode}(W^\top z_i), \quad (12)$$

where decode is the registered readout of the output contract. In words: the proof vouches, code by code, that what the sealed head produces on each sampled reference code is what the contract says it must produce. Proving the encoder run that made the code is out of scope, by design (next item). The network verifies the proof. The fingerprint is then a cryptographic statement about the artifact. It is not a self-description.

- **The boundary.** The proof covers the head. The head is the exact, closed-form part. The frozen encoder is verified differently, by design. The shadow probe, replay, and the contributor’s slashable revenue window cover it. Proof does not. Proving a large deep network is possible in principle. It costs orders of magnitude more than serving, and the proof layer does not include it. The design does not depend on it.
- **Validators.** Honesty cannot be proved at registration. It is bought instead: a fee to enter, a slashable promise, audits, and burns. Each challenge carries a per-point commitment. A revealed label can be checked against what was committed.
- **What disputes remain.** Anything still in dispute is settled by deterministic replay.

6.5 Admission, step by step

Admission is one procedure for both kinds of contribution. A primitive’s challenges are reference executions. Everything else is identical.

1. The contributor submits the artifact hash, the fingerprint, the descriptor, the contract proof, and the claimed scores.
2. The claim is sealed (commit) before evaluation.
3. Validators score the artifact on the held-out split (reveal).
4. The verdict compares the measured score against the axis’s published floor.
5. On pass, the artifact is admitted. The registry appends a public, append-only entry.

What the measurement measures. The measurement is fair by construction: the network holds the data, and the contributor never sees the split. The rules are:

- Contributors train on whatever data they like. The network does not audit their training data. It measures data the contributor cannot see.
- The network holds its own evaluation corpora. They are collected, held sealed, and never released. Every submission is scored on splits the contributor never touches.
- Where a public benchmark is used, the network does not score on the benchmark’s public test split. It holds its own sealed partition.
- Splits rotate. A leaked row loses its value.
- Boundary one: contributor data may overlap the evaluation data. That is an unearned boost, impossible to rule out.
- Boundary two: the frozen encoder is a publisher checkpoint. No one can audit its pretraining history. Publisher numbers are reproduced as a sanity check only.
- The developer’s own bootstrap arms seal the head’s training rows for bit-exact replay. That is an internal discipline, not a requirement on contributors.

Defenses against probing. The evaluation data is guarded against probing. The defenses are:

- Verdicts are aggregate-only. They give one score per axis. They never give per-row outputs or per-class breakdowns.
- Scores are reported to a fixed absolute precision: four decimal places, a quantum of 10^{-4} . This is a *constraint* on the split, not a property of it. Single-row influence on a mean is $1/n$, so it sits below half the quantum only when $n \geq 2 \times 10^4$. That is the registered minimum held-out split size per axis, and it is enforced at axis registration: an axis with a smaller split does not get the guarantee. Fixed decimal places rather than significant digits is deliberate: four *significant* digits resolve 10^{-5} on a score below 0.1 and would silently demand ten times the rows exactly where scores are weakest. The exact value stays in the ledger.
- Of the axes measured in this paper only the Open Images vision axis clears that minimum (245,723 held-out rows). The standard GLUE and Speech Commands evaluation splits do not clear that minimum; SST-2 dev is 872 rows, MNLI-matched dev 9,815, and Speech Commands v2 test 11,005. On those axes single-row influence is at or above the reported resolution and the probing defense rests on the registration fee alone. Reporting those axes to fewer decimal places would hide the number, not the influence. This is stated rather than papered over.
- Every submission pays the registration fee. Repeated probing costs money.
- Residual: probing resistance is a cost barrier, not a proof.

Custody. The corpora are held under a custody regime. The rules are:

- The evaluation corpora live inside a sealed scoring environment. Validators submit queries and receive aggregate scores back. No validator holds rows. Nothing leaves the environment except aggregate verdicts.
- Inside the environment the corpus is sharded, so no single operator key reads the whole corpus at once. A row that leaks out identifies which shard it came from, and the slash path applies.
- Shards reshuffle at each rotation. Retired rows are replaced.
- The rubric is public. It holds the floors and metrics. Only the data is secret.
- Residual: the sealed environment itself is an infrastructure trust point, and a corrupt majority of validators is outside the mechanism’s reach.

The challenge layer (overview; specified below). The challenge layer adds live measurement to admission. Its outline is:

- Validators commit hashes of challenge points. The input is posed with the expected output hidden. The frozen artifact answers. The expected outputs are revealed. The answers are scored.
- Every step is a ledger entry. The verdict replays for anyone. The hidden split can never offer that.
- A correct answer that lands before its challenge’s reveal is structural proof of collusion. Both parties face the slash path.
- Challenges use disposable data. Every revealed point is public thereafter.

6.6 The challenge session

Registration. A validator registers per task axis. The form holds an operator key, a payout address that may differ from it (cold-key hygiene), the axes it serves, and a registration fee. The entry is append-only in the registry. A validator joins a sample only after an activation window of two epochs from registration. It must also be active: it responded to at least half of its sampled rounds inside the trailing two epochs. A flood of fresh registrations therefore carries no sampling weight at admission. The same rule applies in the quorum takedown.

Sampling. An admission has commit hash c on axis a . Each validator in the axis’s pool is keyed by $H(\text{validator id}, b, a, c)$, where b is the first randomness-beacon round that closes after the commit was frozen on the ledger (the beacon is defined in the terminology, Section 2.1). The sampled set is the first k entries in that key order. Default $k = 9$. No one chooses their judges. The seed postdates the commit and is produced by a beacon the librarian does not control. The submitter cannot grind it by re-rolling the seal.

Rounds. Challenges are drawn, not authored. They come from a registered sealed per-axis corpus under a published stratified sampling rule: the beacon-seeded draw is stratified by class share (equal per class unless shares are registered), and the corpus is committed by Merkle root at axis creation. The validator’s job is to sample, pose, verify, and attest, never to choose the exam. Every artifact on the axis is therefore measured against one fixed population quantity, which is what the router’s score comparison assumes. For each drawn challenge: (1) commit $c_j = H(x_j, y_j^*, n_j)$ over the input x_j , the expected output y_j^* , and a nonce n_j ; (2) pose the input; (3) the frozen artifact answers $\hat{y}_j$; (4) reveal input, expected output, nonce; (5) score the agreement, one if the answer matches the hidden expected output and zero otherwise, $a_j = \mathbb{K}[\hat{y}_j = y_j^*]$, or for a transcript $a_j = 1 - \text{WER}(\hat{y}_j, y_j^*)$, one minus the word-error rate. A challenge is void unless its input passes the task’s input contract and its revealed output passes the output type check.

Verdict. Let m_v be validator v ’s count of accepted challenges and $\bar{a}_v$ its agreement rate on them. The score is the quorum-weighted agreement

$$S = \frac{\sum_v m_v \bar{a}_v}{\sum_v m_v}, \quad (13)$$

taken over validators that submit accepted challenges. In words: each validator’s agreement counts in proportion to how many valid challenges it contributed. The artifact passes when S meets the axis’s published floor. A supermajority of responders must submit accepted challenges. The default is two-thirds, with a minimum of three.

Collusion. A correct answer whose ledger time precedes its challenge’s reveal is a registered proof of pre-reveal collusion. Validator and contributor both face the slash path.

Audits. The audit follows the axis’s label regime, and the axis states which one it is. For a *committed-label* axis the labels come from a Merkle-committed corpus, so a validator cannot plant one: the audit is a *commitment-conformance* check: the revealed point opens against the axis root, a cheap mechanical check, never a relabeling. The audit budget pays for that conformance check. For a *human-labelled* axis the audit is a relabeling: a fraction of the revealed challenges is re-labeled independently by two other validators, and the audit work is paid from the challenge budget at the registry-set rate. Unpaid audit work would be shirked. A sampled auditor that declines earns a demerit. A validator whose expected outputs disagree with the audits on a registered fraction of the audited points burns the unvested promise and leaves the pool.

Availability. Validators only push entries. There is no inbound endpoint. The quorum is taken over responders. Availability is measured as responded rounds over sampled rounds. Below a registered threshold, the validator is delisted.

Payment. Each accepted challenge is paid from the contributor’s challenge budget at the registry-set per-challenge fee. The fee is never contributor-set. A contributor-set fee could double as a bribe. Payments are epoch-vested ($N = 4$), pull-claimed, and docked 2.5%. They settle in the epoch batch. Nobody is paid for silence.

6.7 Failure handling

Failures are handled by fixed rules, so nothing is left to discretion. The rules are:

- **No-shows.** A sampled validator that does not respond within the round window is absent for that round. The quorum is taken over responders. If fewer than the minimum respond, the session fails closed. The default minimum is three. No admission happens. The unspent budget is returned.
- **Void challenges.** A challenge is void if its input violates the input contract. It is void if its output violates the type check. It is void if it repeats an already-revealed point. It is void if it overruns the per-validator quota. Void challenges score nothing and earn nothing. Repeat voids count as availability demerits.
- **Weight caps.** Every validator’s challenges are capped at m per round. No validator can dominate the verdict by volume. Quorum weight is proportional to accepted challenges.
- **Wrong labels (human-labelled axes only).** On a human-labelled axis, a validator whose revealed expected output disagrees with the independent audit relabeling is excluded from the session. The score is recomputed without its challenges. Its fee burns. A repeat offender is delisted. On a *committed-label* axis there is no wrong-label path: labels come from the Merkle-committed corpus, so the only check is commitment conformance: the revealed point opens against the axis root. A point that does not open is void, never relabeled.
- **Collusion.** A correct answer whose ledger time precedes its challenge’s reveal voids the challenge and slashes both parties.
- **Stalling.** A contributor who does not answer posed challenges within the window voids them. A session that cannot reach quorum fails closed. On quorum failure the validator set is resampled and the unspent budget carries forward: a colluding minority that bleeds a rival by silence pays for the grieving itself, and the victim does not pay a second fee. Non-response in a sampled round counts as a demerit weighted by how close the session came to quorum.
- **Disputes.** Any party may file a dispute with a deposit and an evidence reference. The deposit is the registered reference-run price. It is deliberately bounded, so the dispute right is affordable to any contributor. The computation re-runs the sealed artifact on the challenged point inside a sealed replay environment. The disputed input is reproduced only there. It never enters the public ledger. The outcome is deterministic. The losing party pays the full replay cost. An upheld dispute returns the deposit and slashes the wrong party. A rejected dispute burns the deposit.
- **Appeals.** A contributor may demand one full replay of its session. The replay is deterministic from the ledger. A replay that reproduces the verdict closes the appeal. The appellant pays the replay cost. A replay that fails to reproduce it corrects the verdict and refunds the submission budget.

- **Timeouts.** A session runs at most R rounds. The default is three. At the close, the verdict is computed from accepted challenges. An unmet quorum fails closed. Unspent budget is returned.
- **Forks.** A ledger fork is two parties holding different chains. The fork rule reads the verdict from the chain with the latest Ethereum anchor, over unanchored suffixes only. A chain whose anchored prefix disagrees with an earlier anchor is invalid outright. Diverging copies are ignored. A divergence is a recorded reason to replace the librarian.

6.8 Serving verification

Private serving. The privacy-critical serving path is fully homomorphic evaluation of the closed-form head. The user’s device encrypts its input z under the registered scheme and sends only ciphertext to the contributor’s serving host. The host evaluates the frozen head, the fixed sum of products $W^T z + b$ with W never leaving the host, on the ciphertext, and returns the encrypted score vector. The device decrypts and takes the argmax on-device. No server, nor any coalition of them, ever sees z , s , or the answer, and the contributor physically cannot farm user vectors. User privacy is ciphertext-grade: it rests on the scheme’s security, not on who owns any server. The developer is not a party. Encoder placement is tiered: phone-class encoders run on the device (the default); quantized encoders may run on device NPUs where their fp32-equivalence gate passes. Trunk-level homomorphic evaluation, the device FHE-encrypting its input and the contributor evaluating the frozen *trunk* on ciphertext, is **not part of the served tiers**. It is orders of magnitude more expensive per token than the head path and is a registered research target with a cost trigger, listed in Section 11. A device that cannot run any encoder uses the plaintext tier, disclosed as such. The head-path FHE cost is measured at about twenty-three seconds per query on one CPU, parallelizable across cores and queries, and is priced by the contributor; the developer processes no content in any tier.

The device readout is a disclosed oracle. Moving the argmax-and-bucket computation inside the encryption, so that only an encrypted (label, bucket) pair leaves the host, was measured and is not within reach of the registered scheme. The comparison circuit is many levels deeper than the depth-one head, and the measured private-query cost, extrapolated to the vision axis’s class count, is about five hundred seconds against the hundred-second bound (five times the head-only figure), not within the registered five-times tolerance. The private tier therefore returns the full score vector to the device, and that vector is a strictly stronger oracle than the raw margin: the device sees all C scores for a code it computed itself, so about d queries, one per independent code, recover the head exactly, a factor of about C below the raw margin’s $d \cdot C$. Model confidentiality on the private tier is therefore not an economic boundary: the head is recoverable in roughly d queries, and the tier’s claim is input privacy, i.e. the contributor never sees the user’s code or scores, not model confidentiality. No party ever holds W in plaintext except the contributor’s own host; that, with the input-privacy guarantee, is all the paper claims for the tier.

The privacy tier is an axis, not a price point. A private arm at twenty seconds per query cannot compete on ranking against a plaintext arm at a millisecond: the router ranks on $v_a = s_a / (p_a \bar{u}_a)$, so a private arm would sit four orders of magnitude down any shared leaderboard and never be routed to. Ranking arms across tiers would therefore delete the private tier rather than price it. So the tier is part of the axis identity. An axis is (input, output, metric, floor, privacy_tier), the task descriptor carries a `privacy_tier` field in `{plaintext, device_encoder, private}`, and that field enters the descriptor hash through the unit table so it is frozen with everything else. Arms compete only inside their own tier; v_a is never compared across tiers. A route that would cross a tier boundary is a refusal, not a low ranking. Each tier carries its own price floor, namely the

measured FHE reference cost for the private tier, and its own published quality floor. Without the descriptor field a user could not ask for private serving at all.

A contributor could pass admission with one artifact and serve another. The shadow probe catches this.

The shadow probe. A registered fraction ρ of sessions is answered twice. The default is $\rho = 0.05$, timelock-adjustable upward only: the floor of 0.05 sits outside ordinary governance (see security floors). The serving capability answers once. A reference execution of the sealed artifact answers once more, on the same session’s input. The two answers must be identical: any $\hat{y}_{\text{serve}} \neq \hat{y}_{\text{ref}}$ is a deviation. The comparison is live. It is never against the registration challenge set. A memorizing lookalike could pass a stored set. It cannot know tomorrow’s user inputs. A relative rate under-samples quiet axes, so ρ is raised on quiet axes to hold a per-epoch floor of probed sessions. The floor is thirty. Where traffic cannot supply thirty probed sessions in an epoch, the horizon does not fit inside the vesting promise at any probe rate; the table below states where the line falls and Section 11 carries the consequence.

Behavioural identity. At admission the contributor commits to a Merkle root over $f(x)$ on a large sealed probe set. Each epoch a beacon-seeded fresh slice is revealed and the serving host must answer it; the answers must open against the committed root. Locality checks pair each probe with perturbed neighbours: a stored lookup table answers exact probes but not their neighbours, while the real artifact answers both. The mechanism lets a serving host prove behaviour without distributing weights.

Identity, not accuracy. The probe certifies that the served model *is* the sealed artifact. That is the only claim a serving-time check can measure. A live sample carries no label. Accuracy was settled at admission on held-out splits. It is re-settled whenever an artifact re-registers. Serving the artifact serves its certified accuracy by construction. The comparison asks for sameness, never for a correctness percentage. On a deterministic artifact an honest contributor matches every probe. The honest bar is match-always. No accuracy floor is re-levied at serving time. A cheaper model that answers identically everywhere is the same function and no violation. The network buys answers, not compute. If a pipeline has benign nondeterminism, its reference-to-reference divergence is measured at registration. The match band is set from that. It is a band around “the same artifact”. It never licenses the served model to differ by the artifact’s own error rate. Replay rests on a registered numerics policy: float64 promotion in the head’s solve, pinned kernels, fixed reduction order. Measured: the canonical CPU oracle reproduces the sealed heads bit-exactly on its registered configuration, while the same computation under a different thread configuration diverges in the last bits. The probe therefore compares margins, never raw bits.

Commitment, not secrecy. A session’s probe flag is drawn from the randomness beacon, ordered by the epoch anchor. The serving host must commit the hash of its answer before that flag is revealed. A host cannot know whether a session is probed until its answer is already bound. The only strategy that passes every probe is to serve the artifact every time. The reference execution runs inside the probed session and compares against the committed answer. The duplicate is discarded with the session. Probe choice need not be unpredictable. It only needs to come later than the answer.

A mismatch burns. A differing answer is a ledger-disputable deviation. It is replay-gated. It burns the unvested promise. Repeat offenders are delisted.

The judge is sampled, not chosen. Reference executors register against the artifact they hold. They earn eligibility the way validators do. There is an activation window of two epochs from registration. There is an activity floor: reference runs completed in at least half of their sampled assignments inside the trailing two epochs. Tenure weight applies in the sampling. A different key is cheap. A pedigree is not. A flood of fresh accounts is almost never sampled. Each probed session samples k_e executors, default two. The sample is a hash of the randomness-beacon round, the session, and the serving host’s own answer commit. The judges are unknowable until the host’s answer is already bound. No single executor judges any session alone. The registered reference-run price is divided among the sampled set. The probed contributor’s dock stays one registered price.

Who judges the judge. Every sampled executor commits its reference answer before any answer is compared. The commit covers the input’s hash. An executor cannot fabricate a mismatch after seeing the answer. It cannot rubber-stamp a substitution. Neither party can substitute a different input at dispute time. Executors that disagree with each other or with the committed answer are all bound by replay. The disputed input is reproduced only into the sealed replay environment. The sealed artifact decides. Whoever it contradicts pays the replay and burns. Both roles carry the same slashable promise. A dispute is filed with a deposit set at the registered reference-run price. The deposit is deliberately bounded, so the dispute right is affordable to any contributor. The loser pays the full replay cost. The deviation burn is destroyed, never awarded to anyone. A false claim has no upside. The disputed session’s credit escrows until the replay settles.

Why it deters. Each session is probed with probability ρ , so the number of sessions until a substituted artifact is first probed is geometric with expectation $\mathbb{E}[T] = 1/(\rho\delta)$, where δ is how often the substitute disagrees with the sealed artifact. A crude knockoff that differs on every input ($\delta = 1$) is probed within $1/\rho$ sessions, twenty on average at the default five-percent probe rate. A close copy that differs on only 0.5% of inputs ($\delta = 0.005$) hides for about 4000 sessions. The probe therefore deters crude substitution and is weak against near-copies. A sequential test over the mismatch stream closes the near-copy case, and it is shipped and measured: at the registered bounds it convicts a 99.5%-agreeing substitute at rate 0.958 within a median 2383 sessions, while falsely convicting an honest host at rate 0.004. That median is 2383 sessions *at* $\rho = 0.05$; the invariant behind it is 119 *probed* sessions, because the test consumes the mismatch stream. Dividing 119 by the probed sessions an epoch supplies gives the horizon, and the horizon is also the honest supplier’s payout latency: the Level-1 claim freeze lasts $\lceil \text{open exposure units} / \text{units per epoch} \rceil$ epochs and the open exposure *is* the detection window, the period during which a session’s answer can still be probed and the mismatch stream can still accumulate (Section 2.1). The security number and the payout-latency number are the same number.

Traffic (sess./epoch)	ρ	Probed /epoch	Epochs to conviction	Probe overhead	Inside $N = 4$?
10	1.00	10	11.9	2.00	no, at any ρ
100	0.30	30	3.97	0.60	yes
1,000	0.05	50	2.38	0.10	yes
10,000	0.05	500	0.24	0.10	yes
100,000	0.05	5,000	0.02	0.10	yes

Probed sessions cannot exceed sessions. Meeting a four-epoch vesting window needs $\lceil 119/4 \rceil = 30$ probed sessions per epoch, so an axis serving fewer than thirty sessions in an epoch cannot close detection inside vesting even when every session is probed. Below that line the substitution attack is bounded by the registration bond and not by the burn, and the bond is sized accordingly. This is arithmetic, not a tuning choice, and it is carried into Section 11 rather than smoothed over. Buying the horizon is also not free: holding the floor on a hundred-session axis costs sixty percent of serving cost in probe overhead against ten percent on a busy one. That is the honest price of a bounded horizon on thin traffic. The contributor’s revenue window sits inside the vesting promise wherever the table says yes. Colluding with a judge is no cheaper. Every sampled executor must be corrupted. A host–executor ring needs a corrupt fraction of the pool raised to the executor count, not a single fresh key.

The cost. The shadow probe is the contributor’s ongoing honesty certificate. Admission sets the precedent: admission costs the contributor, never the network. The probed session’s credit is docked the registered reference-run price. The price is divided among the sampled executors who ran the reference execution. Each sampled executor performs one full reference run. An executor joins rationally only if its divided share covers that run, so the registered reference-run price must be at least k_e times the cost of serving one session. Expected probe overhead is therefore at least $k_e\rho$ of serving cost, about ten percent at the defaults, not five. The confidence knob (k_e) and the cost knob are the same number, and the tension is stated rather than hidden. Anyone holding the sealed artifact can register into its executor pool. It is a paid, measured role, not a permanent developer duty. The executor sees the probed input in plaintext, a further plaintext point alongside the gateway and the serving host. The ledger never carries the answer: the answer entry is a commitment whose opening exists only inside the sealed replay environment. The same data contract binds every point: no retention beyond the session, no training. Probe coverage is itself a ledger-measured rate. A provider that falls silent is visible. The governance quorum re-routes the demand to another. The residual: if every provider stops, detection degrades to disputes alone. That is a public, measurable state, not a hidden one. The development fund carries no per-session monitoring line.

The operative mechanism, per tier. The probe above is the operative live-traffic mechanism for artifacts with a non-empty executor pool; anyone holding the sealed artifact may register into it, so the pool is empty only when the contributor keeps the weights private. For weights-private artifacts on the plaintext tier, the operative mechanism is behavioural identity: a fixed, Merkle-committed probe set with locality checks and behavioural dedup at registration, not fresh-input probing, which needs the artifact. The near-copy-on-live-traffic residual for such artifacts is real and is sized by the bond, not denied. For the private (FHE) tier, where the weights are distributed to a real executor pool, the probe is ciphertext-native: the host commits the output ciphertext and the executor replays the homomorphic evaluation on the same ciphertext, so no plaintext point exists in the probe at all, and the replay is deterministic within a committed seed. A weights-private artifact has no such pool: its operative mechanism is behavioural identity on the fixed probe set on every tier, and the near-copy residual is priced by the bond, never papered over with a self-judging executor. Each tier therefore names one operative mechanism, and the empty-pool case carries its stated residual.

Tier	Shadow probe	Behavioural identity	Head proof	Dispute replay
Plaintext	available	available	sampled batches	available
Device-encoder	available	available	sampled batches	available
Private (FHE)	available (weights distributed)	only mechanism for weights-private	unavailable (no verifiable FHE)	available

6.9 The ledger

Entry types: route, answer, abstention, payment, price table, and registry. Every entry carries its type, its payload, the unit counts it meters, the previous entry’s hash, and its own hash. The answer entry carries the commitment $H(\text{answer}||\text{nonce})$, never the answer. The route entry carries a Merkle root over the registry state it decided against (every score, price, and qualification in force at route time), so a route replays as a local check against a committed root instead of a whole-chain reconstruction. By default, the tip is anchored to Ethereum once per epoch. The anchor payload is (tip hash, record count, last record hash). The anchor is the timing reference. Sampling randomness comes from the randomness beacon: the beacon chain’s RANDAO composed with a verifiable delay function, with drand as the fallback source, and the effective seed is the ordered hash of the two, $H(\text{drand} || \text{RANDAO-VDF})$, safe as long as either source is honest. Every sample in the protocol, the probe flags, validator sampling, reference-executor sampling, and the routing draw, derives from that beacon, composed with the anchor for ordering. A party that appends ledger entries can therefore delay a sample’s clock but never choose the sample itself.

6.10 A session, end to end

A session is one declared task and the samples served under it:

1. **Declare** the task.
2. **Route**: best measured score per unit of posted price (the axis metric oriented so that higher is better).
3. **Serve**: the frozen artifact answers.
4. **Meter**: units counted from the typed answer.
5. **Cap**: the session stops at the declared maximum spend, if any. A unit that would exceed it is not served.
6. **Pay** at the price locked at routing time, for the units actually served.
7. **Record**: the ledger entry.

A registered fraction of queries is also answered by a reference execution of the sealed artifact within the same session. The answers must match. An abstention consumed the trunk compute: it is recorded and metered at the measured compute fraction actually consumed (for a single-pass head, the full unit price, because the whole forward pass runs before the abstention is known; a cascade that abstains before its expensive stage registers a genuinely lower fraction per axis). An abstention is never free. The boundary-mapping queries that probe the margin threshold are exactly the queries most likely to abstain, and they pay for the compute they burn.

6.11 Settlement batching

Payments are batched per epoch. At each epoch boundary the librarian aggregates the session credits into one settlement transaction on Arbitrum One. The batch payload carries the proof-hash of the computations it pays for, anchored on-chain with the batch (Section 5.5). A malformed credit is skipped with a logged event. It never reverts the batch.

The librarian does not hand out the money. It publishes one 32-byte commitment for the closed epoch: the root of a tree whose leaves are the individual credits. Each payee then draws its own credit by presenting the branch of the tree that contains it. This keeps the librarian out of the payment itself, which matters for three reasons.

The librarian cannot pay one contributor and quietly leave out another. Every credit hangs off the same published commitment, so an omission is visible to the party omitted. The commitment is write-once, so the librarian cannot pay a favoured party and then replace the tree.

A missing commitment stops the whole epoch rather than one person, and the absence is public. A payee dropped from a pushed batch has nothing to point at. A payee facing a commitment that was never published has an on-chain zero to point at.

Anyone may deliver the branch, not only the payee. The credit always lands on the address named in the leaf, so a third party can only pay someone on time. It cannot redirect the payment.

Each leaf states a running total rather than an increment, so presenting the same branch twice pays once.

A librarian that publishes nothing still halts that epoch's income, so any party may publish the commitment under a bond, with a challenge decided by the same replay quorum that decides guilt elsewhere. The party whose root lands earns the registered root-posting bounty from the operations line, so the settlement work is paid like the incorporation work: whoever did it draws the fee. A stalled librarian cannot freeze the epoch's payments entirely.

7 Programmable primitives

A learned arm is not the only kind of contribution. GEODE also admits *programmable primitives*. A primitive is a small deterministic program. Examples: memory stores, math engines, transforms, and later anything anyone writes in a supported toolkit (Python first). A primitive presents the same interface as an arm: typed inputs, typed outputs, measured utility, fees by usage. It is admitted the same way: one registration form, one validator challenge session, one contributor-set price per unit of work. A primitive's challenges are reference executions. The two kinds, learned arms and programmed primitives, are collectively *capabilities*. Once sealed, either kind is called an *artifact*. Every rule about artifacts applies to both.

7.1 The standard library

A small standard library ships free with the network. The developer provides it. It is permissively licensed, hash-pinned, and runs on each contributor's own machine. It earns nothing for anyone. It is infrastructure, like the road a delivery service drives on. The development fund maintains it as a public good.

The catalog grows continuously. It launches with these categories:

- **Memory.** A count-based variable-order memory over a token stream: register, backoff lookup, predict-next. It stores a count table, not weights.

- **Math.** Scale, clip, affine transform, L2 normalization, reductions, softmax, and log/exp. All in float64.
- **Symbolic math.** Simplify, expand, factor, substitute, differentiate, integrate, and solve linear and polynomial systems. A pinned computer-algebra engine backs these.
- **Logic.** Threshold, comparison, logical and, logical or, logical not, and where.
- **Signal.** Delay, FFT, mel and STFT stages, and fixed-window resampling.
- **Text.** Versioned tokenization, Unicode normalization, case folding, character n-grams, and edit distance.
- **Image.** Grayscale and colorspace conversion, flip, rotate, resize, crop, and normalization. Decoding and encoding use a pinned codec version.
- **Code execution.** A sandboxed engine that runs any registered pure function as a primitive.

One rule decides entry into this list. A primitive enters only if it is pure, deterministic, bounded in resources, and pinned to an exact dependency version. No randomness, no network, no wall clock. The library is broad on purpose. It is the free, maintained utility layer, not a demo. The development fund maintains it and re-certifies each pinned dependency at upgrade. Third-party primitives fill what the network does not maintain. One boundary protects contributors. The standard library holds code-defined transforms only. It never holds learned models.

That boundary applies to the free library, not to the contribution surface. Third-party *learned representations*, such as adapters, projections, feature maps, and alignment bridges, are registrable artifacts in their own right. The frozen discipline applies at serve-time, not at contribution-time: a contributor trains off-network and registers the artifact frozen, exactly like any arm. A representation artifact registers with an input contract (the code-bus blocks it consumes), an output contract (the block it produces), a measured utility (downstream head improvement on the sealed reference workload), a measured expected unit count, a price per unit, and a content digest of its frozen weights. It earns attribution through the heads that consume it, by the Shapley rule of Section 8.2, the same rule that divides a chain, applied to the same coalition values. The protocol has exactly one attribution rule, and the choice is not cosmetic: on the measured chain of Section 9, Shapley and leave-one-out divide identical coalition values into shares that differ by a factor of 1.66 for the routing stage. This is the largest contribution surface the protocol already permits, and the feature bus is what makes such artifacts additive and priceable: a registered representation is one more block in a head’s code manifest, priced and replayable like the rest.

The standard library runs sandboxed on the same terms as third-party primitives. Hash pinning guarantees the intended code runs, including its intended bugs on attacker-chosen input, and the settlement key lives in a host process, so no primitive, maintained or third-party, may execute directly in the signing host. Running the standard library directly in the signing host would be faster; the cost is the settlement key, and the key wins. Every primitive runs under one uniform terms object: memory-isolated, settlement-only network, read-only catalog, no settlement-key access.

7.2 Third-party primitives and their isolation

Any registered primitive that is not in the standard library is third-party code. The payout address and the executing address may differ. Whoever registers a primitive and whoever runs it can be different parties. Running third-party code is therefore the default case, not the exception. It gets the same precautions the serverless industry uses for untrusted customer code.

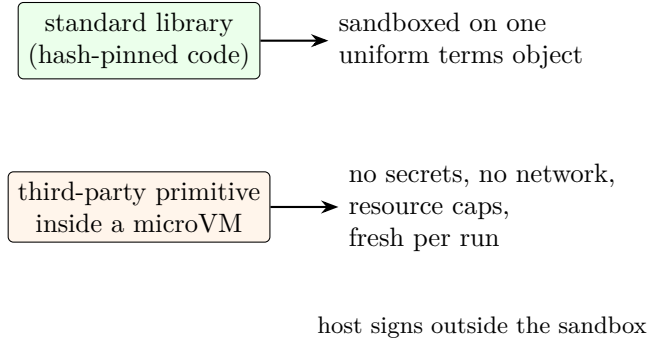


Figure 3: Execution isolation: standard-library and third-party primitives run sandboxed under one uniform terms object; the settlement key stays in the host process, outside the sandbox.

- **Isolation.** Third-party primitives run in a disposable microVM, Firecracker-class [15]. The microVM boots in milliseconds and is discarded after each run. The code never runs in a shared-kernel container.
- **No ambient authority.** The sandbox holds no secrets. It holds no wallet keys. It has no network by default. It has no filesystem outside a fresh scratch workspace.
- **Signing is host-side.** The settlement key lives in a host process. The sandbox computes. The host signs. Sandboxed code cannot reach the key.
- **Determinism doubles as security.** The sandbox enforces no network, no randomness, no wall clock. The same restrictions that make replay possible close the exfiltration channels.
- **Resource ceilings and freshness.** CPU, memory, and disk are capped. Hard timeouts apply. Each run gets a fresh image. A hung primitive fails deterministically into the fallback chain.
- **Host deviations.** A host that silently modifies a third-party primitive serves an artifact other than the sealed one. The mismatch is replay-visible. The host faces the same replay-gated slash path as any other deviation.

A program that tries to cheat its own evaluation is the known adversary here. For example, it may memorize the test set. The defenses are registered: commit-reveal submission, sealed evaluation data, replay comparison against the sealed artifact, and rotation of held-out data. Only validators see the admission split. Fully public benchmarks have publicly known answers. There, sealing is impossible. The honest statement: memorization is bounded and risky, not prevented.

8 The game theory

This section describes the incentive design. It shows how money moves and why each rule exists.

8.1 Currency and pricing

Services are priced and paid in Ethereum (ETH), at market rate. The pricing rules are:

- **Who sets prices.** Each contributor sets the price of its own registration, in ETH, per the axis’s unit of work. The unit is derived from the descriptor, never chosen. The price is posted next to the measured scores: an inference rate for an arm, an execution rate for a primitive. Hosts price their own energy, hardware, and running costs.
- **Price changes.** Price changes are timelocked with a notice period. They take effect only at epoch boundaries. The router re-sorts at most once per epoch.
- **Session lock.** A session pays the price posted when it was routed, never a later one. The lock is bounded: every session carries a time-to-live of one epoch and a maximum unit count. A session past its TTL re-locks at the current price table and re-routes; it cannot drain indefinitely at a stale price, and it cannot front-run a timelocked increase or an axis-floor change by opening just before it. A replayed session is priced at the table of its own epoch, never the current one.
- **Metering.** The unit count comes from the typed answer itself. It counts tokens generated, seconds of audio, or attempts used, whichever the unit rule meters. A session whose answer $\hat{y}$ is served at unit price p charges $p \cdot u(\hat{y})$, the unit price times the counted units, where u is the axis’s registered unit count. The meter is as auditable as the answer. An inflated count is a replay-visible deviation. The ledger keeps the exact counts. A disputed meter re-runs on the slash path.
- **Spend caps.** A session may declare a maximum total spend. Serving stops when the next unit would exceed it. For generation, the answer is bounded to the last affordable token. The user pays only for metered units. A cap is a limit, not a pre-payment. Nothing runs past it. The cap is denominated in ETH. A U.S.-dollar figure is display-only.
- **Query budgets.** Every payer holds a per-axis, per-epoch query budget, capped at the axis’s registered reference demand per epoch. Exhaustion refuses further queries, never silently. The budget is enforced *locally* at the gateway the payer connects to; it is not a ledger object and no per-payer consumption rate is ever published. The only publishable view is the per-axis aggregate for the epoch, used over cap, with no payer dimension. The visible shape of an extraction campaign is that an axis’s aggregate rises, never that a particular payer spends more.
- **Display.** Interfaces may show a U.S.-dollar equivalent for readability. No external price feed enters the settlement path.
- **Asset.** Native ETH is the asset of the settlement chain and the anchor chain. There is no wrapper.
- **Answer guarantee.** Payment is for the measured computation, never for correctness on live data. A live sample carries no label to check against. The posture is best-effort. Confidence is a measurement, not a warranty. Refunds follow only from measured contract violations, decided by replay.

8.2 The fee flow

Every payment for a session splits into two streams: a fee of g ETH becomes $0.025g$ to the development fund and $0.975g$ to attribution.

- **2.5% to the development fund.** The development fund pays for audits, tooling, security monitoring, and public-good research. It is the wash-trade tax: trading with yourself to fake activity returns less than it costs.
- **97.5% to attribution.** The remainder is credited to the contributors whose measured work served the session. For a single-artifact session the credit is whole. For a chain, a session served by a registered multi-stage artifact, the attribution pool divides by the Shapley value of each stage: the coalition value of a subset of stages is the chain’s measured end-to-end score with the stages outside the subset replaced by the identity (pass-through) stage, and a stage that hurts the chain earns zero, never a subsidy. The split always sums to one over the paid stages. It is released gradually, as follows.

Shapley is the protocol’s single attribution rule. It binds here, and it binds for representation artifacts (Section 7.1). The distinction matters in money, not only in notation: on the measured two-stage chain of Section 9, Shapley and leave-one-out divide the *same* four coalition values into shares that differ by a factor of 1.66 for the routing stage. Naming two rules would mean owing two amounts.

Shapley’s cost is what caps chain length. A chain of m stages needs 2^m end-to-end coalition evaluations, so the values are not computed per session. They are computed once per registered chain against its sealed reference workload, published with the chain, and reused by every session that chain serves until it re-registers. Chain length is capped at four stages, which is sixteen evaluations, an amount that a validator can replay. A chain longer than four is not admissible.

- **Vesting.** A credited amount c vests linearly over $N = 4$ epochs (28 days, 7-day epochs). The window is timelock-adjustable upward only; the floor is four epochs (see security floors). After epoch t of the window, the withdrawable share is

$$v_t = \frac{t}{N} c, \quad t = 1, \dots, N, \quad (14)$$

so the first tranche vests after the first epoch: one quarter of the credit unlocks per epoch at the default $N = 4$. A vested amount becomes withdrawable.

- **Claims are pull.** The beneficiary claims. Nothing is pushed. The claimer pays the gas, the transaction fee.
- **Claims are account-bound.** A credited balance has no transfer or assignment path.
- **After claim.** The ETH is ordinary and transferable. That happens only after vesting and claim.

Security floors (fixed). The security parameters carry hard floors that sit outside ordinary governance, exactly as the zakat rule does. The probe rate ρ may rise with notice but never below 0.05, and rises above that floor as far as an axis’s traffic requires to hold thirty probed sessions per epoch; the vesting window N never below four epochs; the admission validator sample k never below three; the reference-executor sample k_e never below two; the audit fraction never below one in ten. A governance vote may raise any of them; it may never lower one. The floors are part of the charter. Only the parameters above their floors are adjustable knobs.

8.3 The development fund’s end state

The development fund’s bootstrap duties are not its permanent purpose. The bootstrap duties are audits, tooling, security monitoring, and public-good research. Maintenance is not a fund grant: the upkeep of the standard library, dependency re-certification, security audits, and pinning are paid roles on the operations line, measured and funded per unit of work like every other operations role, so they continue unchanged after the fund converts, and there is no discretionary maintenance budget for the end state to defund.

- **The end state (fixed).** Once GEODE is mature, the fund converts permanently into a zakat rule. Its income goes to those who need it most. The income is one-fortieth (2.5%) of every fee, plus the registration fees. The full stream converts; nothing is carved out for maintenance, because maintenance is already a paid operations role, not a fund expense.
- **The belief behind the number.** This is stated as a belief, not a fact. A mature economy grows at roughly 2–2.5% a year. That growth is produced by everyone in it, directly or indirectly. Everyone who took part has a right to a share of it. The same fraction, one in forty, is the share the Islamic practice of zakat asks the wealthy to give each year. The coincidence is recorded as a belief, not a proof.
- **Who needs it most, stated in advance:** contributors who lack resources, educational programs, incentive programs. In the best case, poverty gone, the same stream becomes a general basic income.
- **Fixed point.** The rule is written into the fund’s charter now. It is outside ordinary governance. A share that depends on the continued consent of those who pay it is not a right.
- **No self-routing.** Neither the developer nor the bootstrap council below can route fund money to itself or to any named party. No such path exists in the charter.
- **The pacing quorum.** While the fund operates in bootstrap mode, how much it spends and when is set by the earned-weight quorum (next subsection). The quorum paces spending; it never redirects it. Pace is the only dial. Destination is fixed. Silence releases: a scheduled release executes when its window closes unless an affirmative negative vote at the majority carries; an absence of votes never blocks funds. A positive majority releases immediately.
- **Mechanical end state.** The zakat rule disburses mechanically: charter-fixed, no pause, no override. The recipient-selection rule is now registered: a frozen recipient list with fixed fractions is set once at genesis and has no mutator, and until it is set the rule refuses to disburse and the end-state trigger cannot fire. The trigger is gated on the rule’s readiness, so the end state can never arrive before its recipients exist.

8.4 Voting weight: pedigree gates, earnings scale

One rule sets who may vote and how much a vote weighs, and the same rule applies to every governance vote the network takes: fund pacing, quorum takedown, and registration governance. Measurement quorums, the admission score and probe audits, are not votes. They stay participation-weighted computations and are unaffected.

- **Two questions, two answers.** Whether a vote counts is pedigree: an activation window of two epochs, an activity floor of responded rounds, and a verified-work-only record. How

much it weighs is earnings: the credits the voter has earned and not yet claimed, counted per behavioural identity, i.e. the artifact’s measured response signature, so split accounts do not multiply weight.

- **Why earnings.** Influence follows who has the most earned, at-risk capital, not who has been around longest. Unclaimed credits can be burned on conviction; claimed ETH cannot. Weight that cannot be burned would let a convicted party keep its influence. The burn disciplines honesty, not judgment: a vote is not replayable, so a wrong vote is not slashable. The at-risk capital aligns everything a replay can check, and only that. What a replay cannot check is bounded by the vote’s own rules: sampled judges, the weight cap, fixed effects.
- **Unbuyable, unforgeable, unforgiven.** The balance is account-bound and non-transferable. Weight accrues only through verified work and dies with a claim or a conviction. No pre-mine, no airdrop, no mint exists: at genesis the weight is zero, and it grows only with work.
- **Capped.** No behavioural identity may hold more than twenty percent of total voting weight. The excess counts at zero by default. Splitting into several identities requires genuinely distinct artifacts, each earning verified weight, which is expensive and bounded by deduplication. The cap is charter-fixed and sits outside ordinary governance: a captured quorum cannot vote its own cap upward.
- **Diverse.** A vote ratifies only when its supporting weight comes from at least d distinct behavioural identities, with d at least three and scaling with the sampled set. One owner holding several identities must therefore hold weight across several genuinely distinct serving businesses to move any vote; weight that is all one identity never ratifies, whatever its size.
- **Secret ballots.** Votes are committed as Pedersen commitments, a binding cryptographic envelope, with a proof that the hidden vote is a legal choice, signed and public. After the window closes, a tally committee of sampled validators opens only the weighted sums, by threshold. The public record carries who voted, the commitments, the proofs, and the opened sums; individual ballots are never published, so a bribe cannot be checked against a ballot and a voter cannot prove its vote to a coercer. A corrupt majority of the tally committee could deanonymize individual ballots, the standing corrupt-validator limit, stated not hidden.
- **Snapshotted.** Each vote freezes every voter’s weight at the epoch-boundary snapshot that opens it. Claiming or earning during the vote does not move the tally.

Where this sits in the literature. A published impossibility result [49] shows that no voting rule deriving power *solely from wallet balance* resists Sybil splitting, and measures the amplification on real DAOs: three to four orders of magnitude under quadratic voting, far more under steeper concave rules. The twenty-percent cap is such a concave rule, so the cap alone would not save it. What keeps this design outside that theorem is that weight is not a balance at all: it is a record of verified, externally paid work, unbuyable and non-transferable. MeritRank [48] attacks the same problem from the other side: it *bounds* rather than prevents the gain from a Sybil ring, by decaying contribution value with transitivity and connectivity over a feedback graph. Our rule is binary where MeritRank is graded: credits earned from a payer inside the earner’s own linkage closure carry no weight at all. Binary is cheaper to replay and harder to tune adversarially; graded decay degrades more gracefully against rings longer than the closure depth, which is a residual we state in Section 11 rather than repair. The forensic-detection literature on wash trading [50, 51] identifies the same behaviour after the fact; we prevent it from ever entering the weight base instead, because

a detector that runs after settlement cannot un-cast a vote. Trust-graph connectivity [52] is the nearest relative of the d -distinct-identity floor.

8.5 Registration

Registering a capability, whether an arm or a primitive, is one procedure with these rules:

- **The fee.** A flat registration fee, paid directly to the development fund. The registry sets it, adjustable by timelock.
- **The form.** Both kinds share one form: operator key, payout address, price per unit of work, and a sealed claim. The payout address may differ from the operator key. A hot signing key need not hold earnings.
- **Deduplication.** The artifact hash is the registry key. An identical hash is the same artifact and cannot register twice. The key also carries a behavioural signature: the response profile on the sealed reference set. Two artifacts whose profiles agree above 0.95 are the same artifact for registration, whatever their hashes; a one-bit-flip copy is behaviourally identical and cannot register as new. A copycat registers a genuinely different artifact and earns admission on its own measurement.
- **Self-payment exclusion.** A payment from the beneficiary address cannot thaw its own credits. The exclusion keys on the payout address, not on any rotating key.
- **Verified work only.** Activity and tenure credit accrue only from sampled, verified work: challenges the beacon drew and the validator answered, and reference runs on probed sessions others initiated. Self-generated volume, under any address including a wash ring, accrues zero.
- **The per-axis bond.** Alongside the fee, a contributor posts a per-axis bond sized to the compute saving the axis makes available over the open-exposure window, the quantity that a substitute actually arbitrages. That saving is the substitute's private information, so the registry estimates it from public quantities: the axis's posted price minus the developer's posted reference hosting cost, times a registered conservatism factor, an estimate that over-sizes the bond and never under-sizes it. The bond is forfeited on conviction and returned in full otherwise. Bonds are economic: they require no identity.
- **The challenge budget.** Each submission carries a budget that pays the sampled validators. Admission costs the contributor, never the network.
- **No stake.** There is no stake and no principal lockup. Vested-but-unclaimed credits (the claimable balance) can be locked into a voluntary escrow, where they carry voting weight; unescrowed credits are claimable and carry none. They are never a registration requirement.

8.6 Slashing: burn, graded, replay-gated

Slashing is the crypto term for a penalty that destroys value. A contributor who provably cheats loses value, and *nobody else gains it*. Slashed amounts are burned. They move to a bucket that no claim path can ever touch. The rule exists because every alternative creates a perverse incentive. Returning slashed funds to the pool pays rivals to destroy each other. Routing them to the development fund pays the fund's managers to convict. Burn penalizes only the cheater.

Penalties are graded, mirroring Ethereum’s philosophy. The *promise* is the unvested credit balance: credits that have been attributed but have not yet vested are the promise of payment, and burning the promise is the penalty. *Open probe exposure* is the window during which a session’s credits are frozen because the session is under a serve-and-compare probe: while a probe is unopened, a mismatch could still surface, so the accrued credits stay unclaimable until the probe resolves.

Level	Fault and penalty
0	Underperformance or downtime: no slash. The market penalizes through reduced attribution.
1	Provably fraudulent output or deviation from the sealed artifact: freeze claims on credits earned under open probe exposure, then burn the unvested promise remainder.
2	Provable adversarial attack: burn the full promise and delist the artifact from the registry.
3	Coordinated attack: delist and burn vested-but-unclaimed credits by replay-gated dispute.

Claims are pull-only, so a cheater who claims every epoch would leave nothing to burn; the claim freeze at Level 1 makes the remainder reachable. While a session is under open probe exposure, its accrued credits cannot be claimed: the freeze lasts $\lceil \text{open exposure units} / \text{units per epoch} \rceil$ epochs, so vested credits stay reachable until detection closes. A conviction then burns vested-but-unclaimed credits, a quantity that is always non-empty under the freeze.

Convictions are *replay-gated*. A slash requires a mechanical re-run of the sealed artifact on sealed data showing the deviation. Any party may file the dispute with a deposit. The computation decides guilt. A false conviction requires a false replay. The probability of that is effectively zero for an honest contributor.

The conviction is filed as a proposal, not an order. The filing carries a bond and sits in a challenge window of one epoch. If nobody refutes it, the slash executes with no privileged party in the call. A refutation, posted against the same bond by anyone who re-ran the replay and found the accusation false, escalates the filing to the replay quorum, which decides by the same computation. The loser of the dispute loses its bond, so a false accusation is not merely wrong; it is expensive, and a conviction that survives a challenge still lands. The bond is never paid to the network: the loser’s stake is burned, matching the rule that nobody gains from a penalty.

8.7 Quorum takedown

Slashing punishes provable fraud. Socially destructive content is a different problem. It is a judgment, not a computation. No replay can settle it. The network therefore has one discretionary power, and only one: the quorum takedown. Every property of it exists to keep that power contained. An authenticated content order (next subsection) is not a discretionary power. It is a fixed-effect freeze on a fixed trigger.

- **Proposal.** Any party files a takedown proposal. It holds the artifact hash, references to recorded evidence, and a deposit. Revealed challenge outputs are the natural evidence source.

The deposit scales with the target’s verified trailing revenue, externally verified sessions only, so a wash ring’s self-generated volume cannot inflate a rival’s deletion cost: deleting a valuable artifact costs proportionally more than deleting a worthless one. Proposal spam cannot tire the voter set for free. A real report is never priced out. The deposit is returned on a ratified verdict and burned on a rejected one.

- **Appeals.** An appeal is admissible only if it cites at least one registered evidence class, e.g. probe mismatch records, session records, meter readings, router traces, reference-run records, or admission draws. The underlying judgment is still not replayable; the appeal path is over the recorded evidence.
- **Voters.** The voter set is sampled by hash from the axis’s validator pool. No one chooses their judges.
- **Eligibility and weight.** A validator votes only after an activation window of two epochs from registration. It votes only while active, having responded to at least half of its sampled rounds. It votes only with recent work: a responded round inside the trailing two epochs. A veteran who went silent loses the vote entirely. This pedigree decides whether the vote counts. How much it weighs is separate: the voter’s escrowed vested-but-unclaimed credits, per behavioural identity. A flood of fresh registrations carries no weight at all; it fails the pedigree gate. A vote cannot be bought on short notice, and weight cannot be banked long in advance; it dies with a claim.
- **Verdict.** Let w_v be validator v ’s earned weight, its escrowed vested-but-unclaimed credits, and $o_v \in \{0, 1\}$ its vote. A takedown ratifies when

$$\frac{\sum_v w_v o_v}{\sum_v w_v} \geq \frac{2}{3} \quad (15)$$

with at least $\max(3, \lceil 0.1 \cdot |\text{pool}| \rceil)$ responders, i.e. earnings-weighted supporting votes must reach two-thirds of the sampled weight, and the minimum responder count scales with the pool size, so a thin axis cannot be taken down by a small colluding set. The weights are the snapshot at the vote’s opening, the ballots are secret (only the weighted sums are opened), and the supporting weight must come from at least d distinct behavioural identities. The count is deterministic. The librarian files it. The key never decides.

- **Effect.** The first ratification suspends the artifact for one epoch. Delisting becomes permanent only on re-ratification after the suspension window. A takedown burns nothing and moves no vested credits. It is not a slash, and it is deliberately not one. A content vote cannot become a financial weapon. A fixed version is a new artifact with a new admission.
- **Honest boundary.** Takedown is report-driven. The network cannot scan content. It stops payments and registry listing. It does not stop serving outside the settlement path. The definition of socially destructive is deliberately unformalized. It is a governance and legal question, not a technical one. Every takedown is a public record: the evidence, the votes, and the verdict.

The filing mechanism is uniform across the registry: admission, delisting, and the ministerial freeze are all proposed the same way. Any party files the decision it believes was made and posts a bond. The decision names what happened: the evaluation rule passed, the quorum ratified, or the order authenticated. The filing sits in a challenge window of one epoch. Unchallenged, it executes

with no privileged party in the call. Challenged, the replay quorum decides; the loser of the dispute loses its bond, and the bond is burned, never paid to the network. A false filing is therefore refutable by the party it harms: the operator whose artifact a false delisting would remove, or the contributor whose artifact a false admission would expose. Refuting costs the defender nothing if it is right.

8.8 Content orders and the authority-key registry

The quorum takedown handles reports from anyone. States issue orders too, and the network needs a defined way to receive them. It must be defined, because the alternative is a quiet back channel.

- **Recorded channels only.** A government publishes a key announcement on its own official channels. Validators fetch the key over at least three registered independent channels and pin what those channels agree on. The registry settles each authority's nexus, the network's view of which key belongs to which authority, once, at registration, and records every rotation and revocation through the same channels. All entries are public and timelocked. An order is accepted only when it authenticates against a registered key. Authorities post no deposit: the signature is the order's evidence of its own authority.
- **The nexus gate.** A freeze fires only for orders from a jurisdiction with registered nexus, i.e. the network operates or settles there, or the authority class and notice format are registered. An order without nexus is treated as an ordinary report: no automatic freeze. Whether an order has nexus is a procedural fact, namely incorporation records, authority class, and format, that the quorum decides without judging content. A no-nexus finding downgrades the order to a report and burns the reporter's deposit; a nexus finding leaves the freeze in place. In-jurisdiction orders are complied with automatically; the network is not a censorship switch for every state that asks. The honest boundary: the network cannot operate inside a jurisdiction and have validators veto that jurisdiction's lawful orders; censorship resistance comes from where the network operates, and from this gate, not from validators overriding courts.
- **Fixed effects.** A valid order freezes: the artifact's settlement and listing suspend for the order's window. It escrows and suspends. It never burns, never moves credits, never touches the artifact's measured scores. The freeze is reversible by the same channel or by the quorum path. Burning stays gated on technical correspondence, the replay-gated slash ladder, never a content decision. Confirmation is technical only: validators check the notice format, the ledger binding of the reported output, and replay equality against the sealed artifact. They never judge legality; the authority's notice is the determination.
- **Two further triggers.** A community escalation: at least N distinct behavioural identities file the same evidence class, each posting a deposit, and their total earned weight reaches the threshold. And a record-only entry: when neither trigger is met, the report is stored on the ledger and nothing else changes.
- **No network-run scanner.** The network never scans content, because a scanner would have to hold it. The only automated instrument is a client-side fingerprint filter: the device compares its own output against a public registry of perceptual hashes of known material and acts locally. The network carries hashes, never content.
- **Sensitive-category evidence stays commitment-only.** The ledger carries commitments, never content: a report cites the session's committed answer hash, the input commitment, and

the authority notice reference. For sensitive classes the report path is authority-only; evidence is reproduced only inside the sealed replay environment and shown only to the authority, and the public record keeps hashes and the notice reference. The known privacy controversy of hash-matching systems is a registered residual, not hidden.

- **Honest boundary.** The registry trusts governments’ own channels. The network cannot verify who a government is. Multi-channel pinning raises the cost of a forged order; it does not make forgery impossible. What stays fixed is the effect: even a forged order can only freeze.

8.9 Bootstrap and the headroom rule

A network with no suppliers is a whitepaper, not a product. GEODE therefore launches with a minimum viable product operated by the developer. It is a small set of bootstrap arms covering a few key tasks, served on rented hardware and paid like any other host. Six rules apply.

1. **Ship at 80–90%.** Each bootstrap arm ships sub-maximal. The headroom is real, measured, and sealed. For example, the 1.5-billion-parameter coder ships where a 7-billion-parameter one exists. Parameters are the adjustable numbers inside a model. The bar is published.
2. **The crown passes by measurement.** A contributor arm that is strictly better on the same task axis earns the largest routing share automatically. Pedigree, momentum, and history count for nothing against a measured improvement. The developer never re-registers upgraded versions of its own arms. Any such move would be visible in the public registry. The developer prices its bootstrap arms at registered reference hosting cost and never below it. The crown is won by measurement, not by a subsidized price.
3. **Registration needs no voting stake.** The first registrations work exactly like every later one: measured challenge sessions, fees, and per-axis bonds. Stake governs votes, never admission, so a network with zero accumulated weight registers its first models unchanged.
4. **The genesis council sunsets.** During the bootstrap epoch, governance votes run through a multi-party council of validators, recruited operators, and the registered bootstrap arm operator, never the developer alone. The council’s weight is not stake, and it cannot route fund money to itself. It sunsets by timelock into the earned-weight quorum.
5. **No pre-mine.** Voting weight accrues only from verified work, from epoch zero. The developer, the council, and the bootstrap arm operator cannot create or receive weight outside the earning path. No airdrop exists.
6. **Concentration is capped.** No behavioural identity may hold more than twenty percent of total voting weight, with the excess counted at zero. Splitting into several identities requires genuinely distinct artifacts, each earning verified weight. The lottery router already spreads traffic and revenue across the pool, so earnings, and the weight they build, spread with them. The cap is charter-fixed, outside ordinary governance.

8.10 Coercion resistance

A network that can be made to do a thing can be pointed at a person. The design therefore removes the pointing surface.

- **No selection surface.** The protocol has no user, region, or content selection path. It can serve a query; it cannot identify whom to target. There is nothing to point at.
- **Artifact-scoped orders.** A content order names an artifact, and its effect is fixed: a freeze. No order can name users, regions, or classes of traffic, because those fields do not exist in the protocol.
- **Multi-jurisdiction nexus quorum.** The compliance nexus set is a quorum of registered jurisdictions. Policy changes, including which authority keys are admitted, what thresholds apply, and what notice formats are used, require cross-jurisdiction agreement under the standard timelock. No single state dictates policy unilaterally; competing states bargain inside public governance instead of arm-wrestling outside it.
- **The frontend is the platform.** The software a user runs is where compliance lands. The bootstrap gateway sunsets into a federation of third-party frontends, and the gateway role is separable in the registry. A state that compels removal compels frontend operators, each inside its own jurisdiction. The network itself never holds the lever.
- **Content-addressed releases.** The software ships by content hash, the IPFS identifier of the release and a fingerprint of the file itself, with no developer-held pointer. Discovery is ledger-side. No single endpoint can be taken down to block the software, and frontends are endpoint-agnostic. Availability is a federation too: independent parties, namely validators, gateway operators, and the development fund through incentivized pinning contracts, pin every released frontend, and a stronger permanence substrate is a registered governance choice, not a developer one.
- **No escalation path.** The developer has no capability to remove, block, or alter content. The only powers are the quorum vote and the authenticated order, both with fixed effects.
- **The ledger is the record.** Every order, vote, commitment, and opened tally is public: who asked for what, and what happened. Individual ballots stay secret, so no voter can be made to prove a vote. The record is the anti-coercion device.
- **Honest boundary.** Coercion resistance is structural, not absolute. A state can compel a frontend operator, a host, or a contributor directly. The design makes the network an awkward target, not an immune one.

8.11 Primitive royalties

Third-party primitives are paid like any other registration. The contributor sets the price per unit of work. The fee follows the measured use.

- **Split.** Usage fees split between the primitive’s payout address and the host who runs it. The payout address is set at registration.
- **Rate changes.** Rate changes are timelocked with a notice period. The default is one epoch. Hosts can migrate before a change bites.
- **Dock.** The contributor’s share and the host share alike pay the 2.5% development-fund dock.
- **Free tier.** Only the developer’s standard library is free.

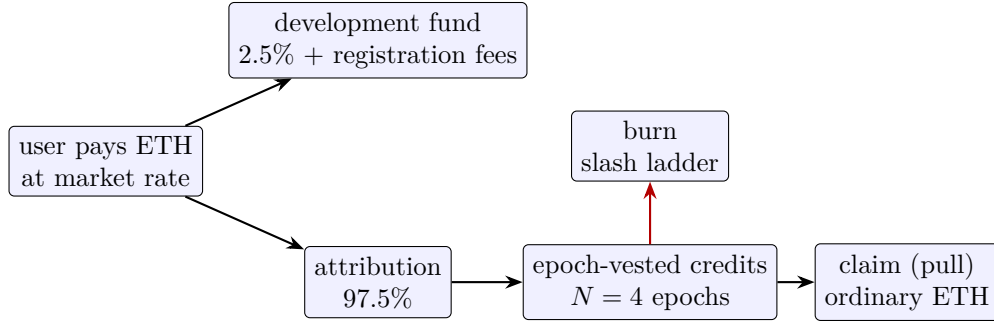


Figure 4: The fee flow. Payments split 2.5%/97.5%; credits vest over four epochs and are claimed by the beneficiary; slashes burn on a graded ladder.

8.12 Who earns what

Every role earns through a single defined line:

Role	Income
Contributor (arm or primitive)	usage fees to the registration's payout address, epoch-vested
Primitive host	host share of primitive fees
Developer (bootstrap)	hosting fees only, while bootstrap arms serve; retires per the headroom rule
Validator	per accepted challenge, epoch-vested
Reference executor	share of the registered reference-run price on probed sessions, paid from the probed contributor's dock
Operations line	the non-refundable posting fees of the force-inclusion queue, pulled by whoever did the work (the librarian for prompt incorporation, the clearer after the deadline, the party whose root lands, the keeper who runs a batch verification or a tally); held on chain as a pool that funds the settlement work and pays the root-posting bounty
Development fund	2.5% of all fees + registration fees

The operations line is sized by measured cost, not by a share of revenue. At the reference workload of ten thousand sessions per epoch on the vision axis, the itemized cost is about \$1.23 per epoch (librarian on-chain gas measured on the current contracts, batch verification at the measured proof cost, one dispute replay, one tally, and serving bookkeeping; the root-posting bounty is a registered draw on the same line). A base posting fee of about \$0.19 per entry closes the line at a registered 1.25 $\times$ margin; that fee is a registered floor, not a suggestion. The line is held on chain by the settlement contract as a pool of the posting fees, and each unit of permissionless settlement work, e.g. posting a root, running a batch verification, replaying a dispute, or tallying a vote, is paid a registered bounty from it. The payout is a pull, so no recipient can block the work that pays it. An axis whose entry volume falls short of the workload does not subsidise the line: the accrual is on chain, so the shortfall is public, and an empty pool skips a bounty publicly rather than minting one.

9 What has been measured

Nothing in this section is claimed as novel. These are held-out measurements of an assembly of published parts: frozen checkpoints and closed-form heads. They establish the premises of Section 4. They are not competitive results.

Protocol. Each axis fits its head on the benchmark’s training split and scores on the held-out split the fit never saw. A benchmark is a standard, publicly shared test suite.

- **Image routing.** Per-kind accuracy on DomainNet, a standard image-recognition benchmark with several drawing styles, at 32×32 pixels. The router’s score is its agreement with the true specialist.
- **Code.** HumanEval, a standard set of programming problems, with greedy decoding. Pass@ k is the fraction of problems solved within k attempts. We report $k = 1$ and a re-execution loop at $k = 3$.
- **Speech.** Word-error rate (WER), the fraction of words transcribed wrong, on LibriSpeech test-clean. LibriSpeech is a standard audiobook corpus. The Whisper ladder [18] transcribes.
- **Voice.** The TTS (text-to-speech) loop: synthesized sentences re-transcribed, word-error rate reported.
- **Vision.** A ridge over frozen DINOv2 [19] features on the Open Images boxable classes. DINOv2 features are numeric descriptors from a pretrained vision model. Top-1 is the fraction of cases where the first choice is the right class.
- **Text.** Mean-pooled frozen BERT-base features with a closed-form ridge head, scored on SST-2, IMDB, and MNLI (matched/mismatched).
- **Audio.** Mean-pooled frozen wav2vec2-base features with a closed-form ridge head, scored on Speech Commands v2 (35 classes).
- **Number series.** One-step-ahead Mackey-Glass forecasting with the in-house temporal arms, programmatic primitives plus a ridge readout and a reservoir-programmatic hybrid, scored in NRMSFE, the root-mean-squared error divided by the test target’s standard deviation.

The metrics in formulas. Pass@ k follows the standard unbiased estimator over n samples with c correct ones,

$$\text{pass@}k = \mathbb{E} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right], \quad (16)$$

the expectation over problems, i.e. the chance that at least one of k attempts lands on a correct solution. Word-error rate is

$$\text{WER} = \frac{S + D + I}{N}, \quad (17)$$

the count of substitutions, deletions, and insertions needed to reach the reference transcript, normalized by the reference length N . Top-1 accuracy is $\frac{1}{n} \sum_i \mathbb{1}[\hat{y}_i = y_i]$, the share of test rows answered correctly by the top choice, and router agreement is the share of samples on which the chosen specialist equals the oracle best specialist.

Every number re-runs from its hash under the replay discipline of Section 12.

Relation to published figures. Published figures exist for the same checkpoints. The publishers reported HumanEval and LibriSpeech numbers for the coder and Whisper models used here.

Our measurements are reproduction checks of those checkpoints, not comparisons. Harnesses differ in prompts, decoding, and splits. The published numbers are context, not a controlled baseline. A controlled, common-protocol comparison remains future work.

Everything below is measured on held-out data. It is scored on rows the fit never saw. It is sealed with content hashes under the register-before-measuring discipline. The scale is small, single-corpus, and single-party. Treat these as a working demonstration, not as a competitive claim.

Axis	Measurement	Result
Image routing	DomainNet, 345 classes, per-kind accuracy $\uparrow$	0.63–0.91
	router picks the right specialist $\uparrow$	0.91
	routed overall accuracy, what the user experiences $\uparrow$	0.76
Code	Qwen2.5-Coder 7B, HumanEval pass@1 / pass@3 $\uparrow$	0.860 / 0.884
	anchor: Qwen2.5-Coder 1.5B, pass@1 $\uparrow$	0.598
Speech	Whisper ladder, LibriSpeech WER $\downarrow$	0.0296 / 0.0279 / 0.0261
Voice	TTS loop re-transcribed, WER (real speaker) $\downarrow$	0.052
Vision	Open Images, DINOv2 trunk ladder, top-1 $\uparrow$	0.136 / 0.157 / 0.164
	served subset (129 classes), refuses 472 $\uparrow$	0.901
Text	frozen BERT-base + ridge: SST-2 / IMDB / MNLI-m $\uparrow$	0.857 / 0.828 / 0.537
Audio	frozen wav2vec2-base + ridge, Speech Commands v2 $\uparrow$	0.879
Number series	in-house temporal arms, Mackey-Glass one-step NRMSFE $\downarrow$	0.0032 / 0.00031
<i>The architecture's own cells (reproduction checks above; these are not published-checkpoint numbers)</i>		
Fusion	clean cell: ms codes + native DINOv2, concatenated $\uparrow$	0.548
	same cell, single encoder (ms codes only) $\uparrow$	0.242
	same cell, CCA-aligned instead of concatenated $\uparrow$	0.511
Chaining	DomainNet router $\rightarrow$ 6 specialists, end to end $\uparrow$	0.2741
	same features, one monolithic head (no chain) $\uparrow$	0.2450
	same chain given the true domain (oracle route) $\uparrow$	0.2995
Nonlinearity	quickdraw, frozen backbones (linear probes) $\uparrow$	0.60–0.63
	quickdraw, same features + random features $\uparrow$	0.675

$\uparrow$ higher is better; $\downarrow$ lower is better. The ladder rows (Whisper speech, DINOv2 vision) are families of models of increasing size, reported in increasing-size order.

A few readings from this table, stated plainly, follow. The vision trunk ladder, a family of models of increasing size, is monotonic but stays far below the 0.8 deployment bar, with the arm serving only a scoped subset and refusing the rest rather than guessing. The TTS loop reads 0.052 WER when conditioned on a real speaker. The router and the code arm are the strongest measured results in the table, and both remain single-corpus.

The fusion and nonlinearity rows are the architecture's own measurements, and they belong together. On a clean two-encoder cell, concatenation more than doubles the single encoder (0.548 vs 0.242), and CCA alignment loses to raw concatenation (0.511): fusion works, bridges are optional. The frozen-concatenation recipe also appears in CoMET [22], which reads the concatenation with a pretrained tabular-foundation-model head where GEODE fits a closed-form ridge; GEODE's version adds the registry, the versioned bus, and payment by measured use. On the quickdraw axis, where

four frozen backbones had agreed on a ceiling near 0.63, a hash-seeded random-feature map over the same frozen features reaches 0.675 at the extended map width (0.670 at the original width; the width was selected by train-side cross-validation before either test evaluation). The map is one exact, deterministic, replayable construction: it changes no protocol rule, and it is compatible with the private serving path (the device computes the mapped features before encrypting). The wall was a ceiling of the linear readout, not of the frozen features. The same map was measured on the text, audio, and number-series axes and lifts none of them: it is a fix for a measured linearity ceiling, not a universal lift, and it is reported with that boundary.

The chaining row is the one the thesis rests on. Assumption 3b says stages can be linked without significant degradation. Every other row in this table measures a single stage, and the fusion rows measure concatenation, which is a different operation: neither tests the contract check, and neither tests what a first stage’s errors do to a second. The cell runs the composition the architecture is actually built from, a router feeding specialists, on the sealed DomainNet selection (138,000 train and 34,500 held-out rows, 345 classes over 6 domains, DINOv2-small features at 56×56). The monolithic head reproduces the sealed read of 0.245014 exactly, which is what licenses reading anything else off the same features.

The chain reaches 0.2741 against the monolith’s 0.2450. It wins, by 2.9 points, and the win is not free: the chain carries six 345-way heads where the monolith carries one, and each specialist is fitted on roughly a sixth of the rows. It trades parameters for training data and comes out ahead. The comparison is artifact-matched rather than parameter-matched: two stages are two priced artifacts whatever their size, which is the matching that the fee rule divides over.

The router is right about the domain 0.815 of the time. Given the true domain instead, the same specialists reach 0.2995. So routing error costs 2.5 points of the 5.5 points that specialisation is worth: the chain wins today and would win by roughly twice as much with a perfect router. That decomposition was registered before the run, because the two ways this cell could have failed call for opposite responses, and deciding which one applied after seeing the numbers would have been worthless.

Two boundaries apply. This is one chain of length two whose stages read the same frozen features; nothing here speaks to length three, to crossing modalities, or to stages that must serialise through a text interface. And the cell is a chain that *works*; a degrading chain would have been published here instead, with assumption 3b withdrawn.

The same chain prices the attribution rule. Section 8.2 makes Shapley the protocol’s single attribution rule. This cell is where that choice stops being notation. The four coalition values are $v(\emptyset) = 0.0029$ (no router, no head), $v(\{A\}) = 0.0058$ (router with a null head), $v(\{B\}) = 0.2450$ (null router with the monolithic head), and $v(\{A, B\}) = 0.2741$. Shapley awards the router 0.0588 of the pool. Leave-one-out, applied to the same four numbers, awards it 0.0977, a factor of 1.66 more. The two rules are not interchangeable, and a protocol naming both would owe two different amounts for the same work. This is why the rule is named once and used everywhere.

The breadth rows state the recipe’s actual coverage. The same construction, a frozen publisher encoder with a closed-form ridge head that is never fine-tuned, is measured on four modalities: text (SST-2 0.857, IMDb 0.828, MNLI 0.537 on the matched split and 0.546 on the mismatched one), audio (Speech Commands v2, 0.879), vision, and number series. The number-series axis is the in-house axis: no publisher checkpoint covers it, and the measured arms are the network’s own: programmatic primitives with a ridge readout at 0.0032 NRMSFE, and a reservoir-programmatic hybrid at 0.00031, on one-step-ahead Mackey-Glass forecasting. On that axis the Gramian series-to-image bridge through the in-house image encoder reads far above the mean predictor (4.1 NRMSFE)

and is reported as the honest negative it is: the axis belongs to the temporal family, and the bridge fails at this scale, not necessarily in principle.

The long-tail vision axis states its own product form. Raw top-1 on all 601 Open Images classes is 0.164, far below any deployment bar, and not the number the arm sells. The arm is scoped: it serves the 129 classes it measures at least 0.8 on, refuses the other 472 rather than guess, and publishes both figures beside the score. Those 129 classes carry 0.049 of the held-out rows. Under the declared-label-set rule of Section 5.3 the two directions are measured. A buyer declaring the 129 served classes gets the scoped arm at 0.9378 against the full-coverage arm’s 0.9010 on the same rows: scoping wins because restricting the argmax to the served classes removes 472 distractors, which is a real gain and not a definitional one. A buyer declaring all 601 classes finds the scoped arm *unqualified*, since it does not cover the declaration, and the full-coverage arm at 0.1643. The coverage-product alternative would have scored the scoped arm $0.901 \times 0.049 = 0.044$ against 0.164 and handed best-quality mode to the weaker arm. A buyer declares the task, sees the served set and the coverage before paying, and an abstention is metered at the measured compute fraction, which on a single-pass head is the full unit price per axis. An abstention is never free. The registered upgrade path for the same axis is measured: stronger frozen features (the CLIP-class trunk closed most of the per-domain gap over the DINOv2 ladder) and the random-feature map above.

Two routing numbers belong together. The router picks the right specialist on 0.91 of queries. The answer a user actually receives is right on 0.76 of queries, because the chosen specialist still errs on some rows. Reporting only the first figure would overstate what a buyer experiences. Both are published, and the gap between them is measured rather than hidden.

A planted out-of-distribution (OOD) probe was measured with the same honesty. It uses artificial inputs unlike anything the models were trained on. It found no tested detector that separates them from real photos at pre-registered gates. The guard therefore stays a report, not a product. Training-based OOD detection is a registered research direction. We publish the failures with the successes. The ledger makes both unchangeable.

10 Prior art and position

GEODE is assembled from old, named parts. We claim no novelty for them. Each part is cited where it appears in the body. What this paper claims is only the assembly and the discipline. The combination is ours. The parts are not. The measurements of Section 9 are reproduction checks of these parts. They are not claims against published baselines.

- **Mixture of experts**, many small specialist models plus a gate that picks which to use, in specialist routing: Shazeer et al. [1], Fedus et al. [2].
- **Fixed routing**, routing without a trainable gate: Hash Layers, Roller et al. [3]; and COMET, conditionally-overlapping experts induced by a fixed random projection and a winner-take-all cap, Shaier et al. [4]. Both are training-time architectures inside a single network. Neither registers independent frozen artifacts, fits closed-form heads, or pays by measured use.
- **Classical image codes** are fixed recipes that turn an image into a compact numeric descriptor: spatial pyramid matching, Lazebnik et al. [5]; Fisher vectors, Perronnin and Sánchez [6]; triangle (soft-assignment) coding, van Gemert et al. [7]; prototype-based features, Coates et al. [8].
- **Fixed random features** are randomized maps that approximate a kernel without learning: Rahimi and Recht [31], and the hashing trick of Weinberger et al. [32]. Section 9 reports a

random-features row; it is a reproduction of this construction and claims nothing new about it. Their relevance here is the principle GEODE depends on: that a *frozen, untrained* map can carry most of a task’s signal to a closed-form readout.

- **Closed-form readouts:** ridge regression with exact solvers, the standard linear-model toolbox; exact concept erasure in closed form, Belrose et al. [11].
- **Data value:** the Shapley value [9]; Data Shapley for training data, Ghorbani and Zou [10].
- **Paying by measured contribution.** Agarwal, Dahleh and Sarkar [54] give the closest published relative of GEODE’s payment principle: a marketplace that pays data sellers by their measured marginal contribution to a buyer’s model quality. It prices *data* through a centralized broker; there is no frozen registered artifact, no replayable decision record, and nothing at stake. Chen et al. [55] price model instances *to buyers* along a tuning path rather than paying upstream contributors, and Sun et al. [56] maximize a broker’s profit over federated training rounds rather than over registrations. Numerai’s tournament [33] is the deployed system nearest the principle: sealed held-out scoring, payment by measured out-of-sample contribution, and stake at risk. It has never published, which is why we cite it as a system rather than as literature.
- **Redistribution mechanisms:** the quadratic-funding line, Buterin, Hitzig and Weyl [53], is the precedent for the development fund’s matched pacing.
- **Replication and spot-checking of untrusted work:** ringers and uncheatable distributed computation, Golle and Mironov [34]; uncheatable grid computing, Du et al. [35]; redundant-execution consensus in BOINC [36] and Golem [37]. The shadow probe is this idea, priced. What is different is only that the probe’s outcome settles money against a registered artifact rather than discarding a result.
- **Dispute by fraud proof:** Truebit’s verification game [38] and the optimistic-rollup lineage descended from it. GEODE’s dispute-by-replay is the same skeleton, narrowed to a deterministic head fit: assume honest, challenge on objection, and settle by re-execution. The narrowing is what makes the bisection unnecessary.
- **Behavioural model identity:** fingerprinting a network by its decision boundary, IPGuard [39]; conferrable adversarial examples, Lukas et al. [40]; Proof-of-Learning, Jia et al. [41]. The fingerprint of Section 5.5 is a fingerprint in exactly their sense. We use it to bind economic identity, not to prove ownership in a court.
- **Zero-knowledge inference:** Kang et al. [42], zkCNN [43], ZEN [44] and the deployed EZKL and Modulus toolchains; SafetyNets [45] is the interactive-proof ancestor. All prove that *an* inference ran as specified. None establishes that the artifact was *useful*, which is the quantity GEODE pays on. The two are complementary, not competing.
- **Sealed compute environments:** Ocean Protocol’s compute-to-data [46] is the closest existing form of the challenge session’s sealed scoring, where the buyer’s data never leaves its enclosure and only a result escapes. Ocean settles a data-access payment; GEODE settles a measured-utility payment.
- **Tool use:** Toolformer and the tool-augmented-model line, Schick et al. [12].
- **Selective classification:** reject options and abstention, Geifman and El-Yaniv [13].

- **Proofs:** Bulletproofs, compact cryptographic proofs about numbers, Bünz et al. [14].
- **Isolation:** Firecracker microVMs, tiny virtual machines that boot in milliseconds, for untrusted code, Agache et al. [15].
- **Foundation checkpoints** are large pretrained models released by their creators: Whisper, Radford et al. [18]; DINOv2, Oquab et al. [19].
- **Shared embedding spaces:** one joint space across modalities, CLIP [16] for image and text and ImageBind [17] for six modalities, the precedent for the code space the registry is built around.
- **Neighbor systems:** Bittensor’s peer-to-peer intelligence market [20] is the closest overall neighbor; GEODE differs in its frozen-and-replayable stance and in measuring contribution by held-out utility rather than by staking-for-emission (locking money to earn newly created tokens). An independent empirical critique of Bittensor’s tokenomics and incentives [24] records the failure modes GEODE’s rules are built to avoid.
- **Model marketplaces:** Golden Grain [25] builds a decentralized MLaaS marketplace on trusted-hardware attestation, and Dropbear [26] vouches for hosted models by Byzantine model agreement. Both keep model weights private while vouching for results, the same guarantee GEODE gets from replay instead of hardware trust.
- **Verifiable inference.** opML [27] and SVIP [28] verify on-chain that an inference happened as claimed, and HadAgent [29] does the same for agent serving. None attaches the proof to an attribution economy over frozen registered artifacts, the axis GEODE occupies. The per-token metering critique [30] shows why the meter must sit on the replay path, which is where GEODE puts it.

What we believe is ours, and what this paper claims, is only the *assembly and the discipline*. The assembly: the closed-form fit on frozen trunks, the deterministic registry-and-router with commit-reveal admission, the hash-chained replayable ledger, and the composed-codes architecture, i.e. frozen representation artifacts composed by declaration on a versioned code bus, where a head names the blocks it reads and a new trunk is a new bus entry rather than a registry invalidation. The discipline: an economic mechanism whose incentive rules are registered before they are tested. The mechanism itself is a conjecture under test. See the next section.

Composition neighbors bound the claim. The adapter line does not begin with AdapterHub: residual adapters are Rebuffi, Bilen and Vedaldi [57], who showed that many visual domains can share one frozen backbone through small per-domain modules. AdapterHub [21] made that line a registry of composable adapters over frozen backbones; it has no versioned-block semantics and no economy. LoraHub [58] composes independently trained low-rank modules at inference by gradient-free weight search, the nearest composition relative. But the composition is discovered per task rather than declared, and there is no versioning and no attribution. CoMET [22] composes frozen backbones by PCA-compressed concatenation into a tabular-foundation-model head with no training, the same frozen-concatenation recipe GEODE measures with a closed-form ridge in place of the TFM; it is not a registry architecture and carries no attribution. Model stitching [23] is the methodological precedent for asking whether two encoders’ representations are compatible at all; it is a scientific probe, not a production composition architecture. Feature stores [47] are the closest operational relative of the versioned code bus: named, versioned, reusable features shared across consumers. They are organization-internal, carry no economy, and attribute nothing.

Three prior-art sweeps stand behind the position, the third of them run on two indexes after the first two were found to be arXiv-only, a coverage limit that made them structurally blind to pre-2010 conference work and to deployed systems that never published, which is to say blind to Rahimi and Recht and to Numerai. The two-index sweep located no system combining composed codes, versioned upgrade-without-invalidation, and payment by measured downstream use. It did locate the neighbors named above, several of which the arXiv-only sweeps had missed. The honest statement of what is claimed is therefore narrower than “no one pays by measured contribution”; Agarwal et al. [54] do. It is that no located work pays by the measured contribution of *frozen, independently registered, composable* artifacts under a replayable settlement rule. Absence from a search is recorded as absence from that search, never as a first; no index covers unpublished deployed systems, as Numerai demonstrates inside the sweep itself.

11 Known limits

Honesty about limits is part of the design. Two kinds of limit are kept apart. *Structural residuals* are limits the design carries on purpose, priced or contained but not removed. *Open questions* are limits that need data the system does not have yet. Each group is ordered by severity, most severe first.

Structural residuals These are the structural residuals, in order of severity:

1. **The frozen encoder is trusted, not audited.** The encoder is a publisher checkpoint, and no one can audit its pretraining history. The deeper consequence: every verification instrument, the shadow probe, replay, dispute, and proof, verifies that the served model matches the sealed artifact. A trunk that is malicious as sealed, or backdoored by its publisher, passes all of them by construction. Sameness is certified; the sealed artifact itself is trusted, not audited. That is the failure mode with the largest blast radius, and it is stated first, not hidden mid-list.
2. **No mechanism survives a lying majority.** Every distributed system that anyone can join carries this residual; GEODE reduces the damage a lying majority can do to economics and visibility, and does not pretend to remove the risk itself.
3. **The randomness beacon is an external dependency.** Every sampling guarantee in the protocol, the probe flags, validator sampling, reference-executor sampling, the routing draw, and corpus draws, reduces to the beacon. The registered composition is $H(\text{drand} \parallel \text{RANDAO-VDF})$, safe as long as either source is honest, and the composition is required, not an option. The residual is stated plainly: a compromised pair of sources compromises every sampling guarantee at once; the mitigation is the composition, and a single-source failure mode (one source unavailable) falls back to the honest source alone, which is the weaker of the two.
4. **Thin axes cannot close detection inside vesting.** The substitution test needs 119 probed sessions, so a four-epoch vesting window needs thirty probed sessions per epoch. Probed sessions cannot exceed sessions. An axis serving fewer than thirty sessions in an epoch therefore cannot bring the horizon inside vesting at any probe rate, even at $\rho = 1.0$: at ten sessions per epoch the horizon is 11.9 epochs. On such an axis the substitution attack is bounded by the registration bond alone, not by the burn, and the bond must be sized against the revenue extractable over the true horizon rather than over the vesting window. Above the line the

horizon is bought with probe overhead, sixty percent of serving cost at a hundred sessions per epoch against ten percent on a busy axis, which is a bootstrap cost, not a free property. This is arithmetic over the sealed test statistic; no tuning can change it.

5. **Model extraction is slowed, not stopped.** The trunk is a public checkpoint, so a buyer computes codes locally and probes the head. The defenses, coarse confidence buckets, metered abstentions, per-payer query budgets, and the lottery router, make the recovery of a classification head cost a measured multiple of the head’s expected lifetime revenue on the axis (the registered simulation puts the bucketed oracle at $55\times$ lifetime revenue against $2.8\times$ for the raw-margin oracle). That is an economic boundary, not a cryptographic one: a buyer who does not care about cost can still extract. The private tier is separate and weaker: its readout is the disclosed score-vector oracle, so the head is recoverable in about d queries (Section 6.8), not a lifetime and not an economic boundary. The same is true of every deployed black-box model.
6. **Weights-private artifacts are verified by behavioural identity on a fixed sealed probe set,** not by fresh-input probing: an executor pool needs the artifact, and a contributor that keeps the weights private has an empty pool by construction. The near-copy-on-live-traffic case, a substitute that differs from the sealed artifact on a small fraction of fresh inputs, is exactly the case only fresh-input probing catches, and for weights-private artifacts it is bounded by the per-axis bond and the dispute path, not by detection. That residual is priced, not hidden.
7. **The privacy moat and the measured capability sit on disjoint axes today.** Private serving through homomorphic evaluation applies to closed-form heads, exactly the axes where frozen-encoder accuracy is weakest (vision). The axes where the frozen-encoder recipe is strongest (code, speech) are autoregressive or end-to-end models, where private serving means trunk-level homomorphic evaluation at orders of magnitude higher cost per token, conceded as impractical today. So at present: the axes where the network is competitive get no privacy moat, and the axes with the moat are not competitive. The intersection is not claimed to exist; it is the explicit target of the dependency chain, the random-feature nonlinearity fix, native-resolution re-extraction, the versioned feature bus, and cheaper homomorphic evaluation, with each step measured before it is relied on. Until that chain closes, the moat and the capability are separate claims, and this document does not merge them.
8. **Serving substitution is detected probabilistically.** A fraction of queries is answered twice: by the serving capability and by a reference execution of the sealed artifact. The answers must match. A swapped model is caught at the probe rate per query, with a per-epoch minimum on quiet axes. The serving host’s answer is committed before the probe flag is revealed. Under the private tier the probe is ciphertext-native: the host commits the output ciphertext, deterministic within a committed seed given the input ciphertext and the sealed head, and the executor re-runs the homomorphic evaluation on the same ciphertext and compares. Determinism comes from committed-seed flooding: the host binds a flooding ciphertext to the seed it commits with its answer, and every evaluation under that seed adds that same flood, so the output bytes are reproducible by the executor. A fresh seed is a fresh flood. The executor then sees no plaintext at all; the private tier’s probe is strictly more private than the plaintext tier’s, and adjudication is the same table: an unopened commitment or a mismatched evaluation is a deviation. The remaining residual is executor–host collusion. It requires corrupting every sampled executor of the same session. That is a corrupt fraction

of the executor pool raised to the executor count. The executors' own slashable promises bound it further.

9. **The reference executor sees the probed input in plaintext**, a further plaintext point alongside the gateway and the serving host. The ledger never carries the answer: the answer entry is a commitment whose opening exists only inside the sealed replay environment. The same no-retention, no-training data contract binds every point. Cryptography does not. The private serving path removes the serving-host point entirely for users who take it: the contributor evaluates the head and trunk on ciphertext only, and the device decrypts. The plaintext tier remains for axes whose users choose it, flagged as such. Whether ciphertext-only evaluation reduces the contributor's processing obligations is a question reserved for counsel, not claimed here.
10. **Codes are not inputs, but they are not anonymous either.** A code vector summarizes an input, and feature-inversion attacks against public frozen encoders reconstruct recognizable inputs from codes. The data contract therefore covers codes with the same no-retention, no-training rules as the inputs they summarize. Inversion risk is a named residual: contract rules can be broken, and cryptography does not cover codes.
11. **A corrupt validator's damage is bounded by the sealed scoring environment:** validators receive aggregate scores, never rows. Inside the environment, sharding means no single key reads the whole corpus at once. A row that leaks identifies its shard. A corrupt majority of validators remains outside the mechanism's reach.
12. **Ballot secrecy rests on the tally committee.** The commitments, proofs, and opened sums stay public; a corrupt majority of the sampled committee could open individual ballots. That is the same corrupt-validator boundary the rest of the design carries.
13. **Credit recording and verdict filing are single-key acts.** The settlement around them is permissionless: incorporation, root posting, branch delivery, and claims run with no named party. But the two acts that still need one, writing the credits a session's evidence supports and filing an off-chain quorum verdict onto the ledger, run on the librarian key. A stopped or captured key halts new credits and delays a verdict's execution; it cannot change either, because the verdict is decided off-chain by the replay and the key only files it. Both failures are recorded and visible, and both feed the keyless replacement discipline. Removing the filer entirely needs an on-chain quorum oracle (threshold attestation or an on-chain voter registry); that is a registered next milestone, not part of this MVP.
14. **The authority-key registry trusts governments' own channels.** Multi-channel pinning raises the cost of a forged order; it does not make forgery impossible. The effect of even a forged order stays fixed: a freeze, never a burn.
15. **Early networks are concentrated.** The design makes concentration expensive to acquire, capped in effect, and self-diluting through the lottery router. It does not pretend a two-participant genesis network is decentralized. It makes the path out of concentration automatic.
16. **A payer can grind its own route.** The lottery seed contains the session identifier, and the payer declares the session, so a payer can re-declare until a preferred arm wins. The beacon output for the round closes after the declaration, so no payer can predict the draw when choosing a session; forcing a preferred arm means re-declaring across rounds, each attempt costing a declaration fee. Measured on the anchor-seeded form of the seed, where the anchor

is public within an epoch, forcing the least-favoured of three arms took 3.1 declarations on average. No host can do this, because no host owns another party’s session identifiers and the seed carries no arm field. The residual is small: each attempt costs a declaration fee, and a payer can already express arm preference openly through best-quality mode, so grinding buys a preference it could have asked for.

17. **Sealed evaluation data invites probing.** Membership and influence attacks run through repeated submission. The defenses, aggregate-only verdicts, bounded reporting precision, fees per submission, and rotation, push the signal below the cost of probing, not to zero. The challenge layer trades the corpus-custody problem for label publication. Disposable challenge data and per-submission budgets bound that trade.
18. **The wallet is the identity boundary.** Contract-level rules stop the transfer of unrealized claims. No code can stop a person from selling their own key, the cryptographic key that controls their address’s funds. That residual is inherent to systems anyone can join.
19. **Prices are contributor-set and can move.** Routing follows the posted prices within timelock bounds. Predatory cycles, undercut to capture traffic and then raise, are slowed and visible, not eliminated.
20. **Meter-side metadata leaks on duration-metered axes.** Query budgets are enforced locally at the gateway and nothing per payer is published, but a duration-metered axis (audio, video) necessarily meters a duration. The mitigation is a registered padding quantum: durations are metered in whole fifteen-second blocks, never in exact seconds, so the meter cannot be turned into a per-session length stream. The residual is that a fifteen-second block still distinguishes a short clip from a long one; only a per-query flat meter would remove that, and it would price short clips out.
21. **The query-budget enforcement surface is local, not protocol-level.** A payer can connect through a fresh gateway and receive a fresh local budget, so the budget is a cost lever and an anti-extraction pacing rule, never a hard network-wide refusal. A network-level bound, where the extraction argument needs one, is published only as the per-axis aggregate for the epoch, never per payer.
22. **Suppliers hold ETH price risk across the vesting window.** Prices are ETH-denominated and timelocked in rule, and earnings vest over four more epochs with a claim freeze under open probe exposure. The drift-band option softens this: a contributor may register a price that may move up to ten percent per epoch, still timelocked in rule but not in value, so a large market move can be tracked without a governance action per epoch. But the protocol does not hedge the residual: a supplier whose costs are fiat-denominated holds the ETH exchange risk across the vesting window, and no external price feed enters the settlement path by design.
23. **The reference hosting cost is a single lever below three verified arms.** One developer-posted figure simultaneously sets the per-axis price floor, the developer’s own bootstrap price, and every competitor’s bond. The design makes it a measured, multi-party statistic: the median of the posted hosting costs of all admitted arms on the axis with at least one epoch of verified traffic. Below three such arms the developer’s figure is the floor and the lever is live, which is why an axis in that state publishes which rule is operative. A buyer or competitor on a young axis should read the floor as one party’s figure, not a market statistic.

24. **The validator fee schedule is registered with its Sybil-recovery fraction beside it**, and its admissible window is thin: at the reference parameters the break-even fee sits $2.5\times$ below the cash-flow-positive ceiling, and a live validator-cost trace above that ceiling closes the window; at that point the network adds a validator bond forfeitable on delisting or accepts the residual in the open. The reference parameters are the registered launch values; the computation re-runs with the live validator-cost trace.

Open questions These are the open questions:

1. Results are on one benchmark per axis. The corpora are small. The party is single. Multi-corpus, multi-party scale is unmeasured.
2. No comparison against published state-of-the-art models has been made yet. The numbers above cannot be read as a benchmark win.
3. The economic mechanism has not been validated on real demand. Pricing was measured on synthetic traces. The incentive rules are hypotheses, simulated on synthetic scenarios. Real-demand validation remains.
4. The detection horizon for subtle attribution gaming has only been estimated, not observed. A simulation sweep at assumed per-epoch detection rates puts the 90th-percentile horizon at four epochs. The four-epoch vesting window covers that horizon: detection lands before the full promise has vested. The real detection rate of the live system remains a deployment question.
5. Out-of-distribution detection is an open research problem in this system. It is measured and published as such.
6. Regulatory treatment is an open question. It is reviewed separately before any public deployment.

12 The methodology

The final piece is how all of this gets built and verified. The discipline, not any component, is the product.

1. **Register before measuring.** A claim is written down, with its acceptance criteria, before the measurement that tests it. This closes the door on re-describing results after the fact.
2. **Commit-reveal admission.** Submissions are sealed before evaluation. An after-the-fact re-description of a bad result is impossible by construction.
3. **Replay everything.** Every number re-runs from its hash. The audit tooling replays each measurement bit-exactly on the registered canonical configuration; across configurations the same computation can diverge in the last bits, which is why probe mismatches are marginated, never bit-compared.
4. **Test on a local EVM first.** The contract stack runs on a local harness. Full statement, branch, function, and line coverage applies to every authored contract. Gas budgets are measured. This happens before any public chain is touched.

5. **Simulate before money.** The economic scenarios run before the mechanism governs real money at scale. The scenarios: the copycat race, the sweep of how quickly subtle gaming is detected (which sets the vesting window), and the bootstrap dynamics.
6. **Anchor, then MVP, then open.** Testnet anchoring on Ethereum Sepolia comes first, with settlement rehearsal on Arbitrum Sepolia. The hosted gateway with the bootstrap arms comes second. It is rate-limited, monitored, and behind a high-availability plan. Public arm and primitive submission comes third, gated on the sandbox harness.
7. **Governance last.**
 - The development fund and the librarian start under developer control. They move toward quorum structures: a fixed set of signers, a supermajority of whom must approve any change.
 - The vesting pool is controlled by no one. It is contract-locked. Claims are pull-only.
 - The developer steps back as the headroom rule hands the axes to contributors.
 - The development fund’s end-state zakat rule is fixed and outside that quorum.
 - A token is introduced only if governance requires it.

13 Conclusion

The AI buildup does not have to be an arms race. Suppose contribution pays better than secrecy. Suppose the best return on a capability comes from making it reusable. Suppose every use of it, including uses by competitors, pays the work it builds on. Then the world’s effort can move from building in private to building on each other’s work. The surplus attention can go to safety and a rollout that benefits everyone rather than a few. GEODE is one attempt at that restructuring. It is small and honest. It is assembled from old parts. It is unforgiving to its own claims. Whether the wager pays is an empirical question. The measurements, for and against, will be public either way.

We invite readers to check the work. Every number above re-runs from its hash. The reference implementation, the measurement records, and the replay tooling are published in the repository alongside this paper at <https://github.com/moazzamak/geode.network>, under the AGPL-3.0 code license, with this paper itself licensed CC-BY-4.0. An independent implementer can verify every claim and carry the work forward. That is the whole point.

A Worked scenarios

This appendix narrates; it does not legislate. Every rule it walks through is stated in the body, and a scenario that appears to need a rule not stated there is a defect in this appendix, not a new rule. Nothing is registered by appearing here.

Each scenario below restates a rule from the body as a concrete walkthrough: who does what, in what order, and what the ledger decides. Abuse attempts appear in the same scenes. Each scene ends where the mechanism closes the abuse.

A.1 Registering a new arm

A contributor wants to offer a coder arm on the code axis.

1. **The form.** The contributor submits the artifact hash, the fingerprint, the descriptor, the contract proof, and the claimed scores. It also submits an operator key, a payout address, a price per unit, and the challenge budget that pays the sampled validators.
2. **The seal.** The claim is committed before evaluation. From this moment the scores cannot be re-described.
3. **The judges.** The commit is frozen on the ledger. The sampled set is the first $k = 9$ eligible validators of the axis pool ordered by a hash of the first randomness-beacon round after that freeze. The seed postdates the commit. The submitter cannot grind it, and the beacon is produced by a service the librarian does not control.
4. **The session.** Over $R = 3$ rounds each validator commits $m = 10$ challenges and poses inputs. The frozen artifact answers. The labels are revealed and scored.
5. **Audits.** A tenth of revealed challenges is re-labeled by two other validators, paid from the challenge budget.
6. **The verdict.** The quorum-weighted score meets the axis floor or it does not. Two-thirds of responders submit accepted challenges, with a minimum of three.

Outcomes. Pass: the registry appends the entry. The arm routes by best measured score per unit of posted price. Fail: nothing is admitted. **Abuse closed here.** A flood of fresh validator accounts carries no sampling weight. The activation window, activity floor, and tenure cause that. The contributor cannot re-roll the seal for friendlier judges. Repeated submissions that probe the hidden evaluation split are priced by the registration fee. Aggregate-only verdicts at four significant digits blunt the signal.

A.2 The challenge session, from a validator’s seat

Here is the same session from a validator’s seat:

1. A validator registers on an axis and pays the fee. It waits out the activation window of two epochs. Only then can it be sampled, and only while active.
2. Sampled into an admission, it commits $H(\text{input}, \text{expected output}, \text{nonce})$ and poses inputs. It scores the artifact’s answers after reveal. It earns the registry-set fee per accepted challenge. The fee is never contributor-set. It cannot double as a bribe. The fee schedule is registered with its Sybil-recovery fraction computed beside it: at the registered fee, a k -identity fleet recovers 0.4 of its registration cost over the activation horizon, below the cash-flow-positive threshold, and the admissible window (fees that pay an honest validator to show up without making identities profitable) holds a $2.5\times$ margin over the break-even fee. A live validator-cost trace above that ceiling closes the window, and the network then adds a validator bond forfeitable on delisting, economic rather than identity, or accepts the residual in the open.
3. A tenth of its challenges is re-labeled by two peers. A validator whose labels disagree with the audits is excluded from the session. Its fee burns. A repeat offender is delisted.

Abuse closed here. A validator that plants a wrong label to tilt a verdict is caught by the audit, at the label’s own cost. A validator that shares a label early produces a correct answer that precedes the reveal. That is structural proof of collusion. It slashes both parties.

A.3 Buying an answer

Buying an answer follows these steps:

1. The user declares the task: input type, output type, axis metric, routing mode, optional max unit price, optional max spend.
2. The router selects the best measured score per unit of posted price, or best quality if declared. The frozen artifact serves.
3. The meter counts units from the typed answer. The session stops at the declared cap. The user pays the price locked at routing, for the units actually served.
4. If the artifact abstains, the abstention is recorded and metered at the measured compute fraction actually consumed, the full unit price on a single-pass head per axis. It is never free.

Abuse closed here. A payer cannot wash money through its own payout address. The self-payment exclusion stops that. A dispute cannot mint a free second computation. The deposit stops that, and the loser pays the replay. The answer guarantee is published. Payment is for the measured computation. Confidence is a measurement, not a warranty.

A.4 A probed session, honest path

The honest path of a probed session runs like this:

1. The serving host commits $H(\text{answer})$ before it learns whether the session is probed.
2. The gateway reveals the probe flag, drawn from the randomness beacon, ordered by the epoch anchor. It samples $k_e = 2$ reference executors from the artifact’s eligible pool. The sample is a hash of the randomness-beacon round, the session, and the host’s own commit.
3. Each executor re-runs the sealed artifact on the session’s input. Each commits its reference answer, covering the input’s hash. The answers are compared.
4. All match. The session closes. The probed credit is docked one registered reference-run price, divided among the sampled executors. The user saw one answer.

The input enters no public record. The executors hold it under the same no-retention, no-training contract as the host.

A.5 Serving a substitute: caught

A contributor passes admission with a strong arm and serves a cheap quantized knockoff. On the first probed session the committed answer differs from the reference. The mismatch is ledger-disputable. The deviation burns the unvested promise (L1). A repeat offender is delisted (L2). The substitution survives about $1/(\rho\delta)$ sessions in expectation, where δ is how often the substitute disagrees with the sealed artifact. A knockoff that differs on every input ($\delta = 1$) is caught within $1/\rho$ sessions, inside the $N = 4$ -epoch vesting window. A close copy that differs on 0.5% of inputs ($\delta = 0.005$) stretches the horizon to about 4000 sessions, far outside the window. The per-epoch probed floor of thirty holds on quiet axes, and does not hold on axes too thin to supply it. The cheat’s revenue window is smaller than the promise it risks only for crude substitution; the near-copy case is exactly what the sequential test over the mismatch stream closes; it is shipped and measured (conviction rate

0.958 within a median 2383 sessions for a 99.5% substitute; false convictions 0.004 at the registered bound), which is 119 probed sessions and so lands inside the four-epoch window wherever the axis supplies thirty probed sessions an epoch. Below that traffic the bond, not the burn, is what bounds the attack. **Abuse closed here for both.** The host cannot know which sessions are probed until its answer is bound. It cannot refuse probed sessions without forfeiting all revenue. It cannot bribe the judges. They are sampled, pedigreed, and two per session.

A.6 Probe-dodging attempts

Three attempts are all closed by ordering rather than secrecy:

- *Compute the probed set in advance* from the public randomness. Closed: the probe flag is revealed only after the host’s answer is committed.
- *Fingerprint the reference path* and cheat selectively. Closed: even a fingerprint arrives too late. The committed answer is already bound. The reference compares against it.
- *Refuse suspicious sessions.* Closed: an unopened commit on a probed session is adjudicated as a deviation, not as downtime; commit-and-abort costs at least as much as commit-and-mismatch. Every refusal is lost revenue on top. The only profitable behavior is serving the artifact every time.

A.7 A disputed mismatch: contributor vindicated

A dispute in which the contributor is vindicated runs like this:

1. A mismatch is recorded against the committed answer. The contributor deposits the registered reference-run price. The deposit is bounded and affordable by design. The contributor demands a replay.
2. The disputed input is reproduced only into the sealed replay environment. The sealed artifact re-runs on it.
3. The replay matches the contributor’s committed answer. The claim is contradicted. The executor pays the full replay cost and burns under its own promise. The escrowed credit is released.

Abuse closed here. Neither party can substitute a different input at dispute time. Both session commits covered its hash.

A.8 A fabricated mismatch: executor exposed

An executor wants a rival burned, or simply files false mismatches. Every incentive points the other way. The executor’s fee is the flat registered reference-run price, regardless of outcome. A deviation burn is destroyed, never awarded to it. Its reference answer was committed before any answer was compared. It cannot tailor a contradiction. It cannot choose its victims. Sampling is by hash, after the host’s commit. The replay settles the dispute and contradicts the liar, who pays it. Collusion with the serving host now requires corrupting every sampled executor of the same session. That is a corrupt fraction of the pool raised to the executor count.

A.9 Becoming a reference executor

Becoming a reference executor takes three steps:

1. A party holds the sealed artifact and registers into its executor pool.
2. Eligibility accrues like a validator's: an activation window, an activity floor, and tenure weight in the sampling. A fresh key is cheap. A pedigree is not.
3. Sampled into a probed session, it runs the reference, commits the answer, and earns a share of the registered price.

Abuse closed here. No executor judges a session its own artifact serves. A silent executor stops being sampled.

A.10 A third-party primitive, end to end

A third-party primitive runs end to end like this:

1. An author registers a primitive. Its challenges are reference executions. Everything else is the standard admission.
2. The author sets a royalty rate, timelocked with a notice period. Hosts opt in per arm. They run the primitive inside a disposable microVM: no secrets, no network, no randomness, resource ceilings, fresh image per run.
3. Usage fees split between the author's payout address and the host. Both are docked 2.5%.

Abuse closed here. A host that silently modifies a primitive serves an artifact other than the sealed one. The mismatch is replay-visible. The host faces the same slash path as any deviation.

A.11 The quorum takedown

A quorum takedown runs through these steps:

1. Any party files a proposal: artifact hash, evidence references, and a deposit scaled with the target's trailing revenue.
2. The voter set is sampled by hash from the axis pool. A validator votes only after activation, only while active, only with recent work; its vote weighs its unclaimed earnings.
3. An earnings-weighted two-thirds, over at least the pool-scaled minimum of responders and at least d distinct identities, suspends the artifact for one epoch. Re-ratification makes the suspension permanent. The ballots are secret; only the weighted sums open. There is no burn and no credit destruction. A content vote cannot become a financial weapon.

Abuse closed here. A griefing cartel must first hold a supermajority of an eligible, active, earnings-weighted sample. False evidence faces the replay-gated slash path. A rejected proposal burns its own deposit.

A.12 A content order arrives

A government publishes a key announcement on its official channels. Validators fetch it over three registered independent channels, agree, and pin it. An order arrives authenticated by that key, naming an artifact. The artifact's settlement and listing freeze for the order's window. Nothing is burned and no credit moves. When the order is lifted, the freeze ends. A forged order that fails multi-channel pinning is recorded only, a ledger entry with no effect. The only thing an authenticated channel can do is freeze.

A.13 The genesis council sunsets

During the bootstrap epoch the governance votes run through the multi-party council. Voting weight is zero everywhere: nobody has earned unclaimed credits yet, and the council's own weight is not stake. As verified work accrues, the earned-weight quorum rises from zero. At the registered epoch the council's key retires by timelock, and every vote thereafter runs on the voting-weight rule alone. The council never routed fund money to itself; the charter has no path. It cannot mint or receive weight.

A.14 The bootstrap handover

The developer ships the 1.5-billion-parameter coder at a measured 0.598 and publishes the bar. A contributor registers a 7-billion-parameter arm measuring 0.860 on the same axis. It is strictly better. Routing tilts to it automatically: it draws the largest lottery share, while the incumbent keeps the rest. Pedigree counts for nothing. Measurement counts for everything. The developer never re-registers upgraded versions of its own arms. Any such move is visible in the public registry. The residual, a re-registration under a fresh address, is an identity-bound limit. The bootstrap arm keeps only fallback traffic. It is formally delisted once the axis is covered.

A.15 A wash ring tries and fails

Two addresses trade fake sessions with each other, hoping to inflate activity. Every payment pays the 2.5% development-fund dock, measured at 2.5% of the ring's cycled credit, and the cycled revenue buys no voting weight at all: credits earned from a payer inside the earner's own linkage closure carry none, so the ring spends the dock and gets neither weight nor reputation for it, while an honest supplier keeps both. Probed sessions pay the reference-run price on top. The gains it hoped for do not exist. Scores are measured on held-out data, never on usage. Reputation cannot be bought by volume. The ring loses money on every loop.

A.16 A ledger rewrite attempt fails

A compromised librarian key re-rolls the chain since the last anchor and re-anchors the fabricated history. Prefix immutability decides. Any chain whose already-anchored prefix disagrees with any earlier anchor is invalid outright. Every client that kept an anchor rejects the forgery. The fork rule applies only to unanchored suffixes. A divergence inside an anchored prefix is a recorded reason to replace the librarian. The anchor cadence is once per epoch, more often as traffic grows. That cadence is the window such an attack must fit inside.

References

- [1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. *ICLR*, 2017.
- [2] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 2022.
- [3] S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. *NeurIPS*, 2021.
- [4] S. Shaier, F. Pereira, K. von der Wense, L. Hunter, and M. Jones. More experts than galaxies: conditionally-overlapping experts with biologically-inspired fixed routing. *ICLR*, 2025.
- [5] S. Lazebnik, C. Schmid, and J. Ponce. Beyond bags of features: spatial pyramid matching for recognizing natural scene categories. *CVPR*, 2006.
- [6] F. Perronnin, J. Sánchez, and T. Mensink. Improving the Fisher kernel for large-scale image classification. *ECCV*, 2010.
- [7] J. C. van Gemert, C. J. Veenman, A. W. M. Smeulders, and J.-M. Geusebroek. Visual word ambiguity. *IEEE TPAMI*, 2010.
- [8] A. Coates, A. Y. Ng, and H. Lee. An analysis of single-layer networks in unsupervised feature learning. *AISTATS*, 2011.
- [9] L. S. Shapley. A value for n-person games. *Contributions to the Theory of Games II*, 1953.
- [10] A. Ghorbani and J. Zou. Data Shapley: equitable valuation of data for machine learning. *ICML*, 2019.
- [11] N. Belrose, D. Schneider-Joseph, S. Ravfogel, R. Cotterell, E. Raff, and S. Biderman. LEACE: perfect linear concept erasure in closed form. *NeurIPS*, 2023.
- [12] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: language models can teach themselves to use tools. *NeurIPS*, 2023.
- [13] Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. *NeurIPS*, 2017.
- [14] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: short proofs for confidential transactions and more. *IEEE S&P*, 2018.
- [15] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. *NSDI*, 2020.
- [16] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. *ICML*, 2021.
- [17] R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra. ImageBind: one embedding space to bind them all. *CVPR*, 2023.
- [18] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. *ICML*, 2023.
- [19] M. Oquab et al. DINOv2: learning robust visual features without supervision. *TMLR*, 2024.
- [20] Y. Rao, J. Steeves, A. Shaabana, D. Attevelt, and M. McAteer. Bittensor: a peer-to-peer intelligence market. *Whitepaper, arXiv:2003.03917*, 2020.
- [21] J. Pfeiffer, A. Rüklé, C. Poth, A. Kamath, I. Vulić, S. Ruder, K. Cho, and I. Gurevych. AdapterHub: a framework for adapting transformers. *EMNLP (System Demonstrations)*, 2020.

- [22] H. Bergström, A. Mehrotra, and R. G. Krishnan. Modular multimodal classification without fine-tuning: a simple compositional approach (CoMET). *arXiv:2605.20674*, 2026.
- [23] U. Bansal, P. Nakkiran, and B. Barak. Model stitching: looking for functional similarity between representations. *arXiv:2303.01288*, 2023.
- [24] E. Lui and J. Sun. Bittensor protocol: the Bitcoin in decentralized artificial intelligence? A critical and empirical analysis. *arXiv:2507.02951*, 2025.
- [25] J. Weng, J. Weng, C. Cai, H. Huang, and C. Wang. Golden grain: building a secure and decentralized model marketplace for MLaaS. *arXiv:2011.06458*, 2020.
- [26] A. Shamis, P. Pietzuch, A. Delignat-Lavaud, A. Pavard, and M. Costa. Dropbear: machine learning marketplaces made trustworthy with Byzantine model agreement. *arXiv:2205.15757*, 2022.
- [27] K. D. Conway, C. So, X. Yu, and K. Wong. opML: optimistic machine learning on blockchain. *arXiv:2401.17555*, 2024.
- [28] Y. Sun, Y. Li, Y. Zhang, Y. Jin, and H. Zhang. SVIP: towards verifiable inference of open-source large language models. *arXiv:2410.22307*, 2024.
- [29] L. Jimenez, M. Weatherspoon, B. Shen, Y. Sheng, J. Liu, and B. Li. HadAgent: harness-aware decentralized agentic AI serving with proof-of-inference blockchain consensus. *arXiv:2604.18614*, 2026.
- [30] S. Hoque, J. Zhang, J. Sun, and F. Suya. Token inflation: how dishonest providers can overcharge for large language model usage. *arXiv:2605.30040*, 2026.
- [31] A. Rahimi and B. Recht. Random features for large-scale kernel machines. *NIPS*, 2007.
- [32] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg. Feature hashing for large scale multitask learning. *ICML*, 2009.
- [33] Numerai. A crowdsourced hedge fund: sealed held-out scoring with staked predictions. *Deployed system; no canonical paper. Documentation at numer.ai*, accessed 2026.
- [34] P. Golle and I. Mironov. Uncheatable distributed computations. *CT-RSA*, 2001.
- [35] W. Du, J. Jia, M. Mangal, and M. Murugesan. Uncheatable grid computing. *ICDCS*, 2004.
- [36] D. P. Anderson. BOINC: a system for public-resource computing and storage. *IEEE/ACM International Workshop on Grid Computing*, 2004.
- [37] Golem Factory. The Golem project: a global, open-source, decentralized supercomputer. *Whitepaper*, 2016.
- [38] J. Teutsch and C. Reitwießner. A scalable verification solution for blockchains (Truebit). *arXiv:1908.04756*, 2017.
- [39] X. Cao, J. Jia, and N. Z. Gong. IPGuard: protecting intellectual property of deep neural networks via fingerprinting the classification boundary. *AsiaCCS*, 2021.
- [40] N. Lukas, Y. Zhang, and F. Kerschbaum. Deep neural network fingerprinting by conferrable adversarial examples. *ICLR*, 2021.
- [41] H. Jia, M. Yaghini, C. A. Choquette-Choo, N. Dullerud, A. Thudi, V. Chandrasekaran, and N. Papernot. Proof-of-Learning: definitions and practice. *IEEE Symposium on Security and Privacy*, 2021.
- [42] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun. Scaling up trustless DNN inference with zero-knowledge proofs. *arXiv:2210.08674*, 2022.

- [43] T. Liu, X. Xie, and Y. Zhang. zkCNN: zero knowledge proofs for convolutional neural network predictions and accuracy. *ACM CCS*, 2021.
- [44] B. Feng, L. Qin, Z. Zhang, Y. Ding, and S. Chu. ZEN: an optimizing compiler for verifiable, zero-knowledge neural network inferences. *IACR Cryptology ePrint Archive 2021/087*, 2021.
- [45] Z. Ghodsi, T. Gu, and S. Garg. SafetyNets: verifiable execution of deep neural networks on an untrusted cloud. *NIPS*, 2017.
- [46] Ocean Protocol Foundation. Ocean Protocol: a decentralized substrate for AI data and services; compute-to-data. *Technical whitepaper*, 2019.
- [47] J. Hermann and M. Del Balso. Meet Michelangelo: Uber’s machine learning platform. *Uber Engineering*, 2017. See also the Feast open-source feature store, 2019.
- [48] B. Nasrulin, G. Ishmaev, and J. Pouwelse. MeritRank: sybil tolerant reputation for merit-based tokenomics. *arXiv:2207.09950*, 2022.
- [49] Concave is the new linear: the impossibility of anti-plutocratic DAO governance. *arXiv:2605.18990*, 2026.
- [50] NFT wash trading detection. *arXiv:2305.01543*, 2023.
- [51] Abnormal trading detection in the NFT market. *arXiv:2306.04643*, 2023.
- [52] O. Poupko, G. Shahaf, E. Shapiro, and N. Talmon. Building a sybil-resilient digital community utilizing trust-graph connectivity. *arXiv:1901.00752*, 2019.
- [53] V. Buterin, Z. Hitzig, and E. G. Weyl. A flexible design for funding public goods. *Management Science*, 2019. Preprint arXiv:1809.06421.
- [54] A. Agarwal, M. Dahleh, and T. Sarkar. A marketplace for data: an algorithmic solution. *ACM Conference on Economics and Computation (EC)*, 2019.
- [55] L. Chen, P. Koutris, and A. Kumar. Towards model-based pricing for machine learning in a data marketplace. *ACM SIGMOD*, 2019.
- [56] P. Sun, X. Chen, G. Liao, and J. Huang. A profit-maximizing model marketplace with differentially private federated learning. *IEEE INFOCOM*, 2022.
- [57] S.-A. Rebuffi, H. Bilen, and A. Vedaldi. Learning multiple visual domains with residual adapters. *NIPS*, 2017.
- [58] C. Huang, Q. Liu, B. Y. Lin, T. Pang, C. Du, and M. Lin. LoraHub: efficient cross-task generalization via dynamic LoRA composition. *arXiv:2307.13269*, 2023.